



VERIFAYA TEAM

UVM PROJECT SPI SLAVE

**PRESENTED BY:
Carol Micheal-Ram
Farah Ihab-Slave
Manar Saber-Wrapper**

**PRESENTED TO:
Eng Kareem Waseem**

Table of Contents

1.0	SLAVE.....	3
1.1	SPI Slave	3
1.2	Slave Golden.....	11
1.3	Slave Interface.....	13
1.3	Slave Sequence Item	13
1.5	Slave Sequence.....	15
1.6	Slave Sequencer.....	21
1.7	Slave Configuration Object.....	21
1.8	Slave Driver	22
1.9	Slave Monitor.....	22
1.10	Slave Scoreboard	23
1.11	Slave Coverage.....	25
1.12	Slave Agent.....	26
1.13	Slave Env	28
1.14	Slave Test.....	28
1.15	Slave Top.....	29
1.16	Verification Plan.....	31
1.17	Bug Report.....	31
1.18	Assertion Table	31
1.19	Coverage Report	33
1.20	Questasim.....	37
1.21	Do file.....	39
2.0	RAM.....	39
2.1	Interface.....	39
2.2	SVA module.....	40
2.3	Sequence item	41
2.4	Sequence	42
2.5	Sequencer.....	46
2.6	Configuration Object.....	46
2.7	Driver.....	47
2.8	Monitor	47
2.9	Scoreboard	48

2.10 Coverage	50
2.11 Agent	52
2.12 ENV package	53
2.13 Test	54
2.14 Top	55
2.15 Adjusted RAM design	56
2.16 Golden Model	57
2.17 Verification Plan	58
2.18 Bug Report	59
2.19 Assertion Table	60
2.20 Coverage Report	61
2.21 Questasim	64
2.22 DO File	66
3.0 WRAPPER	66
3.1 Wrapper	66
3.2 Wrapper_Top	67
3.3 Wrapper_Interface	68
3.4 Wrapper_Sequence_Item	69
3.5 Shared_Package	69
3.6 Wrapper_Sequence	70
3.7 Wrapper_Sequencer	71
3.8 Wrapper_Configuration_Object	72
3.9 Wrapper_Driver	72
3.10 Wrapper_Monitor	73
3.11 Wrapper_Scoreboard	74
3.12 Wrapper_Coverage	76
3.13 Wrapper_Agent	77
3.14 Wrapper_Env	77
3.15 Wrapper_Test	78
3.16 Verification_Plan	79
3.17 Assertion_Table	79
3.18 Coverage_Report	80
3.18 Wrapper_Questasim	82
3.19 Do_File	83

3.20 Scr_File	83
4.0 Diagrams	85
4.1 Wrapper	85
4.2 RAM	86
4.3 SLAVE	86

1.0 SLAVE

1.1 SPI Slave

```
module SLAVE (MOSI,MISO,SS_n,clk,rst_n,rx_data,rx_valid,tx_data,tx_valid);
localparam IDLE      = 3'b000;
localparam WRITE     = 3'b001;
localparam CHK_CMD   = 3'b010;
localparam READ_ADD  = 3'b011;
localparam READ_DATA = 3'b100;

input          MOSI, clk, rst_n, SS_n, tx_valid;
input [7:0] tx_data;
output reg [9:0] rx_data;
output reg      rx_valid, MISO;
reg [3:0] counter;
reg      received_address;
reg [2:0] cs, ns;

always @(posedge clk) begin
    if (~rst_n) begin
        cs <= IDLE;
    end
    else begin
        cs <= ns;
    end
end

always @(*) begin
    case (cs)
        IDLE : begin
            if (SS_n)
                ns = IDLE;
            else
                ns = CHK_CMD;
        end
        CHK_CMD : begin
            if (SS_n)
                ns = IDLE;
            else begin
                if (~MOSI)
                    ns = WRITE;
                else begin
                    if (!received_address) //when equal 0 not 1
                        ns = READ_ADD;
                end
            end
        end
    endcase
end
```

```

        else
            ns = READ_DATA;
        end
    end
end
end
WRITE : begin
    if package wrapper_coverage_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"
import wrapper_seq_item_pkg::*;
import shared_pkg::*;

class wrapper_coverage extends uvm_component;
    `uvm_component_utils(wrapper_coverage)

    uvm_analysis_export #(wrapper_seq_item) cov_export;
    uvm_tlm_analysis_fifo #(wrapper_seq_item) cov_fifo;
    wrapper_seq_item seq_item_cov;

    covergroup cvgrp;
        W_cmd_cov_cp: coverpoint seq_item_cov.cmd iff (seq_item_cov.rst_n &&
~seq_item_cov.SS_n) {
            bins wa_bin = {WA};
            bins wd_bin = {WD};
            bins ra_bin = {RA};
            bins rd_bin = {RD};
        }
        W_rst_cov_cp: coverpoint seq_item_cov.rst_n {
            bins reset_active  = {0};
            bins reset_inactive = {1};
        }
        ssn_cov_cp: coverpoint seq_item_cov.SS_n {
            bins active  = {0};
            bins inactive = {1};
        }
    }

    MISO_cov_cp: coverpoint seq_item_cov.MISO {
        bins miso_0 = {0};
        bins miso_1 = {1};
    }

    cmd_miso_cross : cross W_cmd_cov_cp, MISO_cov_cp {

        // Legal combinations based on your FSM
        bins write_phase = binsof(W_cmd_cov_cp) intersect {WA, WD} &&
            binsof(MISO_cov_cp.miso_0);
    }
end
end
end

```

```

        bins read_phase = binsof(W_cmd_cov_cp) intersect {RA, RD} &&
                           binsof(MISO_cov_cp.miso_1);

        // All other combinations are illegal
        illegal_bins illegal_operations =
            (binsof(W_cmd_cov_cp) intersect {WA, WD} &&
binsof(MISO_cov_cp.miso_1)) ||
            (binsof(W_cmd_cov_cp) intersect {RA, RD} &&
binsof(MISO_cov_cp.miso_0));
    }
endgroup

function new(string name = "wrapper_coverage", uvm_component parent =
null);
    super.new(name, parent);
    cvgrp = new;
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    cov_export = new("cov_export", this);
    cov_fifo = new("cov_fifo", this);
endfunction

function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    cov_export.connect(cov_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
        cov_fifo.get(seq_item_cov);
        cvgrp.sample();
    end
endtask
endclass
endpackage(SS_n)

        ns = IDLE;
    else
        ns = WRITE;
    end
    READ_ADD : begin

```

```

        if (SS_n)
            ns = IDLE;
        else
            ns = READ_ADD;
        end
    READ_DATA : begin
        if (SS_n)
            ns = IDLE;
        else
            ns = READ_DATA;
        end
        //missing default case
    default: ns = IDLE;
endcase
end

always @(posedge clk) begin
    if (~rst_n) begin
        rx_data <= 0;
        rx_valid <= 0;
        received_address <= 0;
        MISO <= 0;
        counter <= 0; //<--- missing
    end
    else begin
        case (cs)
            IDLE : begin
                rx_valid <= 0;
            end
            CHK_CMD : begin
                counter <= 10;
            end
            WRITE : begin
                if (counter > 0) begin
                    rx_data[counter-1] <= MOSI;
                    counter <= counter - 1;
                end else
                    rx_valid <= 1;
            end
            READ_ADD : begin
                if (counter > 0) begin
                    rx_data[counter-1] <= MOSI;
                    counter <= counter - 1;
                end else begin
                    rx_valid <= 1;
                end
            end
        endcase
    end
end

```



```

        received_address <= 1;
    end
end
READ_DATA : begin
    if (tx_valid) begin
        rx_valid <= 0;
        if (counter > 0) begin
            MISO <= tx_data[counter-1];
            counter <= counter - 1;
        end
    else begin
        received_address <= 0;
    end
end
    else begin
        if (counter > 0) begin
            rx_data[counter-1] <= MOSI;
            counter <= counter - 1;
        end else begin
            rx_valid <= 1;
            counter <= 9; //bug detected
            received_address <= 0;
        end
    end
end
endcase
end
end
`ifdef SIM
//assertion 1//
a_reset_outputs_low: assert property (@(posedge clk)
    !rst_n | => (MISO==1'b0 && rx_valid==1'b0 && rx_data==10'b0))
else $error("RESET: outputs not low while rst_n==0.");
c_reset_outputs_low: cover property (@(posedge clk)
    !rst_n | => (MISO==1'b0 && rx_valid==1'b0 && rx_data==10'b0));

//assertion 2//
a_counter_load_in_chk: assert property (@(posedge clk) disable iff(!rst_n)
    (cs==CHK_CMD && SS_n==0) | => (counter==4'd10));
c_counter_load_in_chk: cover property (@(posedge clk) disable iff(!rst_n)
    (cs==CHK_CMD && SS_n==0) | => (counter==4'd10));

a_ssn_rxvalid_write: assert property (@(posedge clk) disable iff(!rst_n)
    (((cs==WRITE) || (cs==READ_ADD)) && (counter==4'd10) && (SS_n==1'b0))
    | => ( ((!rx_valid)[*10] ##1 rx_valid) ##[0:12] (SS_n==1) ))

```

```

    else $error("SS_n not released within 0..12 cycles after rx_valid for
cs=%0d", cs);
c_ssn_rxvalid_write: cover property (@(posedge clk) disable iff(!rst_n)
    ((cs==WRITE) || (cs==READ_ADD)) && (counter==4'd10) && (SS_n==1'b0))
    | => ( (!rx_valid)[*10] ##1 rx_valid) ##[0:12] (SS_n==1) ));

a_ssn_rxvalid_read: assert property (@(posedge clk) disable iff(!rst_n)
    ((cs==READ_DATA) && (counter==4'd10) && !tx_valid && (SS_n==1'b0))
    | => ( (!rx_valid)[*10] ##1 rx_valid) ##[0:23] (SS_n==1) ))
    else $error("SS_n not released within 0..23 cycles after rx_valid in
READ_DATA");
c_ssn_rxvalid_read: cover property (@(posedge clk) disable iff(!rst_n)
    ((cs==READ_DATA) && (counter==4'd10) && !tx_valid && (SS_n==1'b0))
    | => ( (!rx_valid)[*10] ##1 rx_valid) ##[0:23] (SS_n==1) ));

// FSM transition assertions
// IDLE -> CHK_CMD
property prop_idle_to_chk;
    @(posedge clk) disable iff (!rst_n)
        (cs == IDLE && SS_n == 1'b0) | => (cs == CHK_CMD);
endproperty
assert property (prop_idle_to_chk)
    else $error("[SVA] FSM: IDLE -> CHK_CMD transition failed");
cover property (prop_idle_to_chk);

// IDLE hold while SS_n is high
property prop_idle_hold;
    @(posedge clk) disable iff (!rst_n)
        (cs == IDLE && SS_n == 1'b1) | => (cs == IDLE);
endproperty
assert property (prop_idle_hold)
    else $error("[SVA] FSM: IDLE did not hold when SS_n stayed high");
cover property (prop_idle_hold);

// CHK_CMD -> IDLE when frame aborts
property prop_chk_to_idle;
    @(posedge clk) disable iff (!rst_n)
        (cs == CHK_CMD && SS_n == 1'b1) | => (cs == IDLE);
endproperty
assert property (prop_chk_to_idle)
    else $error("[SVA] FSM: CHK_CMD -> IDLE transition failed");
cover property (prop_chk_to_idle);

// CHK_CMD -> WRITE
property prop_chk_to_write;

```

```

    @(posedge clk) disable iff (!rst_n)
        (cs == CHK_CMD && SS_n == 1'b0 && MOSI == 1'b0) | => (cs == WRITE);
endproperty
assert property (prop_chk_to_write)
    else $error("[SVA] FSM: CHK_CMD -> WRITE transition failed");
cover property (prop_chk_to_write);

// CHK_CMD -> READ_ADD
property prop_chk_to_read_add;
    @(posedge clk) disable iff (!rst_n)
        (cs == CHK_CMD && SS_n == 1'b0 && MOSI == 1'b1 && received_address ==
1'b0)
        | => (cs == READ_ADD);
endproperty
assert property (prop_chk_to_read_add)
    else $error("[SVA] FSM: CHK_CMD -> READ_ADD transition failed");
cover property (prop_chk_to_read_add);

// CHK_CMD -> READ_DATA
property prop_chk_to_read_data;
    @(posedge clk) disable iff (!rst_n)
        (cs == CHK_CMD && SS_n == 1'b0 && MOSI == 1'b1 && received_address ==
1'b1)
        | => (cs == READ_DATA);
endproperty
assert property (prop_chk_to_read_data)
    else $error("[SVA] FSM: CHK_CMD -> READ_DATA transition failed");
cover property (prop_chk_to_read_data);

// WRITE holds while SS_n low
property prop_write_hold;
    @(posedge clk) disable iff (!rst_n)
        (cs == WRITE && SS_n == 1'b0) | => (cs == WRITE);
endproperty
assert property (prop_write_hold)
    else $error("[SVA] FSM: WRITE did not hold while SS_n remained low");
cover property (prop_write_hold);

// WRITE -> IDLE
property prop_write_to_idle;
    @(posedge clk) disable iff (!rst_n)
        (cs == WRITE && SS_n == 1'b1) | => (cs == IDLE);
endproperty
assert property (prop_write_to_idle)
    else $error("[SVA] FSM: WRITE -> IDLE transition failed");

```

```

cover property (prop_write_to_idle);

// READ_ADD holds while SS_n low
property prop_readadd_hold;
    @(posedge clk) disable iff (!rst_n)
        (cs == READ_ADD && SS_n == 1'b0) | => (cs == READ_ADD);
endproperty
assert property (prop_readadd_hold)
    else $error("[SVA] FSM: READ_ADD did not hold while SS_n remained low");
cover property (prop_readadd_hold);

// READ_ADD -> IDLE
property prop_readadd_to_idle;
    @(posedge clk) disable iff (!rst_n)
        (cs == READ_ADD && SS_n == 1'b1) | => (cs == IDLE);
endproperty
assert property (prop_readadd_to_idle)
    else $error("[SVA] FSM: READ_ADD -> IDLE transition failed");
cover property (prop_readadd_to_idle);

// READ_DATA holds while SS_n low
property prop_readdata_hold;
    @(posedge clk) disable iff (!rst_n)
        (cs == READ_DATA && SS_n == 1'b0) | => (cs == READ_DATA);
endproperty
assert property (prop_readdata_hold)
    else $error("[SVA] FSM: READ_DATA did not hold while SS_n remained low");
cover property (prop_readdata_hold);

// READ_DATA -> IDLE
property prop_readdata_to_idle;
    @(posedge clk) disable iff (!rst_n)
        (cs == READ_DATA && SS_n == 1'b1) | => (cs == IDLE);
endproperty
assert property (prop_readdata_to_idle)
    else $error("[SVA] FSM: READ_DATA -> IDLE transition failed");
cover property (prop_readdata_to_idle);

`endif
endmodule

```

1.2 Slave Golden

```
module slave_golden (MOSI,MISO,SS_n,clk,rst_n,rx_data,rx_valid,tx_data,tx_valid);
input MOSI;
input clk, rst_n, SS_n, tx_valid;
input [7:0] tx_data;
output reg [9:0] rx_data;
output reg rx_valid, MISO;
reg [3:0] counter;
reg received_address;
reg [2:0] cs, ns;
localparam [2:0]
    IDLE      = 3'b000,
    WRITE     = 3'b001,
    CHK_CMD   = 3'b010,
    READ_ADD  = 3'b011,
    READ_DATA = 3'b100;

always @(posedge clk) begin
    if (~rst_n) begin
        cs <= IDLE;
    end
    else begin
        cs <= ns;
    end
end
always @(*) begin
ns = cs;
case (cs)
    IDLE: begin
        if (SS_n) ns = IDLE; else ns = CHK_CMD;
    end
    CHK_CMD: begin
        if (SS_n) ns = IDLE;
        else if (~MOSI) ns = WRITE;
        else if (!received_address) ns = READ_ADD;
        else ns = READ_DATA;
    end
    WRITE:      ns = (SS_n ? IDLE : WRITE);
    READ_ADD:   ns = (SS_n ? IDLE : READ_ADD);
    READ_DATA:  ns = (SS_n ? IDLE : READ_DATA);

    default:    ns = IDLE;
endcase
end
```

```

end
always @(posedge clk) begin
    if (!rst_n) begin
        rx_data          <= 10'b0;
        rx_valid         <= 1'b0;
        MISO             <= 1'b0;
        counter          <= 4'd0;
        received_address <= 1'b0;
    end else begin
        case (cs)
            IDLE: begin
                rx_valid <= 1'b0;
            end
            CHK_CMD: counter <= 10;
            WRITE: begin
                // Receive 10 bits MSB-first into rx_data[9:0]
                if (counter > 0) begin
                    rx_data[counter-1] <= MOSI;
                    counter <= counter - 1;
                end else rx_valid <= 1'b1;
            end

            READ_ADD: begin
                // Receive 10-bit address
                if (counter > 0) begin
                    rx_data[counter-1] <= MOSI;
                    counter <= counter - 1;
                end else begin
                    rx_valid          <= 1'b1;
                    received_address <= 1'b1;
                end
            end

            READ_DATA: begin
                if (tx_valid) begin
                    rx_valid <= 0;
                    if (counter != 0) begin
                        MISO <= tx_data[counter-1];
                        counter <= counter - 1;
                    end else begin
                        received_address <= 1'b0;
                    end
                end else begin
                    // Receive 8 bits from MOSI
                    if (counter != 0) begin

```

```

        rx_data[counter-1] <= MOSI; // fill bits [7..0]
        counter          <= counter - 1;
    end else begin
        rx_valid <= 1'b1;
        counter  <= 9;
        received_address <= 1'b0;
    end
end
end
endcase
end
end
endmodule

```

1.3 Slave Interface

```

interface slave_if(clk);
input bit clk;
logic rst_n, SS_n, MOSI, rx_valid;
logic MISO, tx_valid;
logic [7:0] tx_data;
logic [9:0] rx_data;
//golden model outputs
logic MISO_ref;
logic rx_valid_ref;
logic [9:0] rx_data_ref;
endinterface

```

1.3 Slave Sequence Item

```

package slave_sequence_item;
import uvm_pkg::*;
`include "uvm_macros.svh"
class slave_seq_item extends uvm_sequence_item;
`uvm_object_utils(slave_seq_item)
rand bit clk, rst_n, rx_valid, rx_valid_ref;
rand bit [7:0] tx_data;
rand bit tx_valid;
rand bit[9:0] rx_data_ref, rx_data;
rand bit MISO, MISO_ref;
rand bit MOSI, SS_n;

```

```

rand bit [2:0] cmd; // 3'b000, 001, 110, 111 only

//////////constraints//////////
constraint rst_c {rst_n dist {1:/90, 0:/10};}
constraint SS_n_c {
    solve tx_valid before SS_n;
    (!tx_valid) -> SS_n dist {1:=1, 0:=12};
    (tx_valid) -> SS_n dist{1:=1, 0:=22};
}
rand bit [10:0] MOSI_array [0:3];
constraint MOSI_c {
    foreach (MOSI_array[i]) MOSI_array[i][10:8] inside
{3'b000,3'b001,3'b110,3'b111};

    // all-different on the 3-bit headers:
    MOSI_array[0][10:8] != MOSI_array[1][10:8];
    MOSI_array[0][10:8] != MOSI_array[2][10:8];
    MOSI_array[0][10:8] != MOSI_array[3][10:8];
    MOSI_array[1][10:8] != MOSI_array[2][10:8];
    MOSI_array[1][10:8] != MOSI_array[3][10:8];
    MOSI_array[2][10:8] != MOSI_array[3][10:8];
}
//The tx_valid signal to be high in case of read data
constraint cmd_c { cmd inside {3'b000, 3'b001, 3'b110, 3'b111}; }
constraint cmd_map_c { rx_data[9:7] == cmd; }
constraint tx_valid_c {
    (cmd == 3'b111) -> (tx_valid == 1);
    (!cmd == 3'b111) -> (tx_valid == 0);
}
function new(string name="slave_seq_item");
super.new(name);
endfunction

function string convert2string();
return $sformatf("%s rst_n=0b%0b rx_valid=0b%0b MISO=0b%0b MOSI=0b%0b SS_n=0b%0b,
tx_data=0b%0b, tx_valid=0b%0b, rx_data=0b%0b, rx_valid_ref=0b%0b, MISO_ref=0b%0b,
rx_data_ref=0b%0b",super.convert2string(),
rst_n,rx_valid,MISO,MOSI,SS_n,tx_data,tx_valid,rx_data,rx_valid_ref,MISO_ref,rx_d
ata_ref);
endfunction

function string convert2string_stimulus();
return $sformatf("rst_n=0b%0b rx_valid=0b%0b MISO=0b%0b MOSI=0b%0b SS_n=0b%0b,
tx_data=0b%0b, tx_valid=0b%0b, rx_data=0b%0b, rx_valid_ref=0b%0b, MISO_ref=0b%0b,
rx_data_ref=0b%0b",

```



```

    rst_n,rx_valid,MISO,MOSI,SS_n,tx_data,tx_valid,rx_data,rx_valid_ref,MISO_ref,rx_
data_ref);
endfunction
endclass
endpackage

```

1.5 Slave Sequence

```

package slave_seq_pkg;
import uvm_pkg::*;
import slave_sequence_item::*;
`include "uvm_macros.svh"

typedef enum bit [2:0] {
    CMD_WA = 3'b000,
    CMD_WD = 3'b001,
    CMD_RA = 3'b110,
    CMD_RD = 3'b111
} cmd_e;

class slave_reset_sequence extends uvm_sequence #(slave_seq_item);
    `uvm_object_utils(slave_reset_sequence)
    slave_seq_item reset_item;

    function new(string name="slave_reset_sequence");
        super.new(name);
    endfunction

    task body;
        // Alternate rst_n asserted/deasserted to guarantee a 50% duty reset pulse
train.
        int unsigned total_cycles = 16;
        int unsigned rst_low_count = 0;
        int unsigned rst_high_count = 0;

        for (int idx = 0; idx < total_cycles; idx++) begin
            bit rst_n_val = (idx % 2) ? 1'b1 : 1'b0;
            reset_item = slave_seq_item::type_id::create($sformatf("rst_item_%0d",
idx));
            start_item(reset_item);
            reset_item.rst_n      = rst_n_val;
            reset_item.SS_n      = 1;
            reset_item.tx_valid = 0;
            reset_item.MOSI      = 0;
            reset_item.tx_data   = '0;

```

```

        reset_item.cmd      = CMD_WA;
    finish_item(reset_item);

    if (rst_n_val)
        rst_high_count++;
    else
        rst_low_count++;
    end

    if (rst_low_count != rst_high_count) begin
        `uvm_error("RST_SEQ",
            $sformatf("Reset sequence imbalance: rst low=%0d, high=%0d",
                rst_low_count, rst_high_count))
    end else begin
        `uvm_info("RST_SEQ",
            $sformatf("Reset sequence issued %0d items with a 50%% duty
cycle",
                total_cycles),
            UVM_LOW)
    end
endtask
endclass

class slave_main_sequence extends uvm_sequence #(slave_seq_item);
    `uvm_object_utils(slave_main_sequence)

    function new(string name="slave_main_sequence");
        super.new(name);
    endfunction

    task automatic drive_frame(cmd_e cmd_sel,
        int rd_tx_mode = -1,
        bit [7:0] forced_tx_data = 8'h00,
        bit use_forced_tx = 0,
        bit use_forced_payload = 0,
        bit [7:0] forced_payload = 8'h00,
        bit use_forced_lsb = 0,
        bit forced_lsb_value = 0,
        bit use_rx_payload = 0,
        bit [7:0] forced_rx_payload = 8'h00,
        bit use_rx_lsb = 0,
        bit forced_rx_lsb = 0,
        int drop_tx_valid_from = -1);

        int          low_len;
        bit [10:0] bits;

```

```

bit [7:0]  frame_tx_data;
bit [7:0]  payload;
bit [7:0]  rx_payload;
slave_seq_item gap;
slave_seq_item beat;
int i;
static bit lsb_flip = 0;
bit rd_tx_en;
bit is_read_cmd;
bit rd_rx_mode;

low_len      = (cmd_sel == CMD_RD) ? 23 : 13;
is_read_cmd  = (cmd_sel == CMD_RA) || (cmd_sel == CMD_RD);

payload = (cmd_sel == CMD_RA) ? 8'hA5 : $urandom();
if (use_forced_payload)
    payload = forced_payload;

bits[10:8] = cmd_sel;
bits[7:0]  = payload;

if (use_forced_lsb) begin
    bits[0]  = forced_lsb_value;
    lsb_flip = forced_lsb_value ^ 1'b1;
end else begin
    bits[0]  = lsb_flip;
    lsb_flip ^= 1'b1;
end

frame_tx_data = use_forced_tx ? forced_tx_data : $urandom();
if (cmd_sel == CMD_RD) begin
    if ((rd_tx_mode == 0) || (rd_tx_mode == 1))
        rd_tx_en = rd_tx_mode;
    else
        rd_tx_en = $urandom_range(0,1);
end else begin
    rd_tx_en = 1'b0;
end

rd_rx_mode = (cmd_sel == CMD_RD) && (rd_tx_en == 0);
rx_payload = $urandom();
if (use_rx_payload)
    rx_payload = forced_rx_payload;

gap = slave_seq_item::type_id::create("gap");

```

```

start_item(gap);
    gap.rst_n      = 1;
    gap.SS_n       = 1;
    gap.MOSI       = (cmd_sel == CMD_RD) ? 1'b1 : 1'b0;
    gap.tx_valid   = 0;
    gap.tx_data    = '0;
    gap.cmd        = cmd_sel;
finish_item(gap);

for (i = 0; i < low_len; i++) begin
    beat = slave_seq_item::type_id::create($sformatf("low_%0d", i));
    start_item(beat);
        beat.rst_n      = 1;
        beat.SS_n       = 0;
        beat.cmd        = cmd_sel;
        beat.tx_valid   = (cmd_sel == CMD_RD) ? rd_tx_en : 1'b0;
        if ((cmd_sel == CMD_RD) &&
            (rd_tx_en == 1'b1) &&
            (drop_tx_valid_from >= 0) &&
            (i >= drop_tx_valid_from)) begin
            beat.tx_valid = 1'b0;
        end
        beat.tx_data    = frame_tx_data;
        if (i == 0) begin
            beat.MOSI = is_read_cmd ? 1'b1 : 1'b0;
        end else if (rd_rx_mode && (i >= 11) && (i < 19)) begin
            int rx_idx;
            bit rx_bit;

            rx_idx = i - 11;
            rx_bit = rx_payload[7 - rx_idx];
            if (use_rx_lsb && (rx_idx == 7))
                rx_bit = forced_rx_lsb;
            beat.MOSI = rx_bit;
        end else if (use_forced_lsb && (i == 10)) begin
            beat.MOSI = forced_lsb_value;
        end else if (i < 11) begin
            beat.MOSI = bits[10 - i];
        end else begin
            beat.MOSI = 1'b0;
        end
        finish_item(beat);
    end
endtask

task automatic drive_abort_chk();

```

```

slave_seq_item gap;
slave_seq_item first;
slave_seq_item abort;

gap = slave_seq_item::type_id::create("gap_abort");
start_item(gap);
    gap.rst_n      = 1;
    gap.SS_n       = 1;
    gap.MOSI       = 0;
    gap.tx_valid   = 0;
    gap.tx_data    = '0;
    gap.cmd        = CMD_WA;
finish_item(gap);

first = slave_seq_item::type_id::create("abort_chk_first");
start_item(first);
    first.rst_n    = 1;
    first.SS_n     = 0;
    first.MOSI     = 1'b1;
    first.tx_valid = 0;
    first.tx_data  = '0;
    first.cmd      = CMD_RA;
finish_item(first);

abort = slave_seq_item::type_id::create("abort_chk_release");
start_item(abort);
    abort.rst_n    = 1;
    abort.SS_n     = 1;
    abort.MOSI     = 0;
    abort.tx_valid = 0;
    abort.tx_data  = '0;
    abort.cmd      = CMD_RA;
finish_item(abort);
endtask

function automatic cmd_e pick_cmd();
    int sel;
    sel = $urandom_range(0,99);
    if      (sel < 25) return CMD_WA;
    else if (sel < 50) return CMD_WD;
    else if (sel < 75) return CMD_RA;
    else          return CMD_RD;
endfunction

virtual task body();

```

```

// deterministic coverage seed:
drive_frame(CMD_RA);
drive_frame(CMD_RD, 1, 8'hF0, 1);

drive_frame(.cmd_sel(CMD_RA),
            .use_forced_payload(1),
            .forced_payload(8'h3C),
            .use_forced_lsb(1),
            .forced_lsb_value(1'b1));

drive_frame(.cmd_sel(CMD_RD),
            .rd_tx_mode(0),
            .forced_tx_data(8'h0F),
            .use_forced_tx(1),
            .use_forced_payload(1),
            .forced_payload(8'hD3),
            .use_rx_payload(1),
            .forced_rx_payload(8'hA7),
            .use_rx_lsb(1),
            .forced_rx_lsb(1'b1));

drive_frame(.cmd_sel(CMD_RA),
            .use_forced_payload(1),
            .forced_payload(8'h96),
            .use_forced_lsb(1),
            .forced_lsb_value(1'b0));

drive_frame(.cmd_sel(CMD_RD),
            .rd_tx_mode(0),
            .forced_tx_data(8'hF0),
            .use_forced_tx(1),
            .use_forced_payload(1),
            .forced_payload(8'h4C),
            .use_rx_payload(1),
            .forced_rx_payload(8'h53),
            .use_rx_lsb(1),
            .forced_rx_lsb(1'b0));

// Force tx_valid low late in the frame so counter drops to zero and
exercises
// the READ_DATA rx path where (counter > 0) evaluates to false.
drive_frame(.cmd_sel(CMD_RD),
            .rd_tx_mode(1),
            .drop_tx_valid_from(21));

```

```

drive_abort_chk();

drive_frame(.cmd_sel(CMD_WD),
            .use_forced_payload(1),
            .forced_payload(8'h01),
            .use_forced_lsb(1),
            .forced_lsb_value(1'b1));

drive_frame(.cmd_sel(CMD_WD),
            .use_forced_payload(1),
            .forced_payload(8'h00),
            .use_forced_lsb(1),
            .forced_lsb_value(1'b0));

repeat (90) begin
    cmd_e cmd_sel;
    cmd_sel = pick_cmd();
    drive_frame(cmd_sel);
end
endtask
endclass
endpackage

```

1.6 Slave Sequencer

```

package slave_sequencer_pkg;
import uvm_pkg::*;
import slave_sequence_item::*;
`include "uvm_macros.svh"
class slave_sequencer extends uvm_sequencer #(slave_seq_item);
`uvm_component_utils(slave_sequencer)
function new(string name="slave_sequencer",uvm_component parent =null);
    super.new(name,parent);
endfunction
endclass
endpackage

```

1.7 Slave Configuration Object

```

package slave_config_pkg ;
import uvm_pkg::*;
`include "uvm_macros.svh"

```

```

class slave_config extends uvm_object;
  `uvm_object_utils(slave_config)
virtual slave_if slave_vif;
function new (string name="slave_config");
  super.new(name);
endfunction
endclass
endpackage

```

1.8 Slave Driver

```

package slave_driver_pkg;
import uvm_pkg::*;
import slave_sequence_item::*;
`include "uvm_macros.svh"
class slave_driver extends uvm_driver #(slave_seq_item);
  `uvm_component_utils(slave_driver)
virtual slave_if slave_vif;
slave_seq_item stim_seq_item;
function new(string name="slave_driver",uvm_component parent=null);
  super.new(name,parent);
endfunction

task run_phase(uvm_phase phase);
  super.run_phase(phase);
  forever begin
    stim_seq_item=slave_seq_item::type_id::create("stim_seq_item");
    seq_item_port.get_next_item(stim_seq_item);
    slave_vif.SS_n=stim_seq_item.SS_n;
    slave_vif.rst_n=stim_seq_item.rst_n;
    slave_vif.MOSI = stim_seq_item.MOSI;
    slave_vif.tx_valid=stim_seq_item.tx_valid;
    slave_vif.tx_data=stim_seq_item.tx_data;
    @(negedge slave_vif.clk);
    seq_item_port.item_done();
    `uvm_info("run_phase",stim_seq_item.convert2string_stimulus(),UVM_HIGH)
  end
endtask
endclass
endpackage

```

1.9 Slave Monitor

```

package slave_monitor_pkg;

```



```

import uvm_pkg::*;
import slave_sequence_item::*;
`include "uvm_macros.svh"
class slave_monitor extends uvm_monitor;
`uvm_component_utils(slave_monitor)
virtual slave_if slave_vif;
slave_seq_item rsp_seq_item;
uvm_analysis_port #(slave_seq_item) mon_ap;
function new(string name="slave_monitor",uvm_component parent=null);
super.new(name,parent);
endfunction
function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon_ap=new("mon_ap",this);
endfunction
task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
        rsp_seq_item=slave_seq_item::type_id::create("rsp_seq_item");
        @(negedge slave_vif.clk);
        rsp_seq_item.rx_data = slave_vif.rx_data;
        rsp_seq_item.rx_data_ref = slave_vif.rx_data_ref;
        rsp_seq_item.rx_valid = slave_vif.rx_valid;
        rsp_seq_item.rx_valid_ref = slave_vif.rx_valid_ref;
        rsp_seq_item.rst_n = slave_vif.rst_n;
        rsp_seq_item.MISO = slave_vif.MISO;
        rsp_seq_item.tx_valid = slave_vif.tx_valid;
        rsp_seq_item.MISO_ref = slave_vif.MISO_ref;
        rsp_seq_item.SS_n = slave_vif.SS_n;
        rsp_seq_item.MOSI = slave_vif.MOSI;
        rsp_seq_item.tx_data = slave_vif.tx_data;
        mon_ap.write(rsp_seq_item);
        `uvm_info("run_phase",rsp_seq_item.convert2string(),UVM_HIGH)
    end
endtask
endclass
endpackage

```

1.10 Slave Scoreboard

```

package slave_scoreboard_pkg;
import uvm_pkg::*;
import slave_sequence_item::*;
`include "uvm_macros.svh"

```

```

class slave_scoreboard extends uvm_scoreboard;
`uvm_component_utils(slave_scoreboard)
uvm_analysis_export #(slave_seq_item) sb_export;
uvm_tlm_analysis_fifo #(slave_seq_item) sb_fifo;
slave_seq_item seq_item_sb;

integer error_count=0;
integer correct_count=0;
virtual slave_if vif;

function new(string name="slave_scoreboard",uvm_component parent=null);
super.new(name,parent);
endfunction

function void build_phase(uvm_phase phase);
super.build_phase(phase);
sb_export=new("sb_export",this);
sb_fifo=new("sb_fifo",this);
endfunction

function void connect_phase(uvm_phase phase);
super.connect_phase(phase);
sb_export.connect(sb_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);
super.run_phase(phase);
forever begin
    sb_fifo.get(seq_item_sb);
    if (seq_item_sb.MISO != seq_item_sb.MISO_ref || seq_item_sb.rx_valid !=
seq_item_sb.rx_valid_ref || seq_item_sb.rx_data != seq_item_sb.rx_data_ref)
begin
        `uvm_error("run_phase",$sformatf("Mismatch: %s | DUT MISO=%0b
rx_valid=%0b rx_data=%0h | REF MISO=%0b rx_valid=%0b rx_data=%0h",
seq_item_sb.convert2string(), seq_item_sb.MISO,
seq_item_sb.rx_valid, seq_item_sb.rx_data, seq_item_sb.MISO_ref,
seq_item_sb.rx_valid_ref, seq_item_sb.rx_data_ref));
        error_count++;
    end
    else begin
        `uvm_info("run_phase",$sformatf("correct
out:%s",seq_item_sb.convert2string()),UVM_HIGH);
        correct_count++;
    end
end
end

```

```

endtask

function void report_phase(uvm_phase phase);
    super.report_phase(phase);
    `uvm_info("report_phase",$sformatf("total successful
transactions:%0d",correct_count),UVM_MEDIUM);
    `uvm_info("report_phase",$sformatf("total failed
transactions:%0d",error_count),UVM_MEDIUM);
endfunction

endclass

endpackage

```

1.11 Slave Coverage

```

package slave_coverage_pkg;
import uvm_pkg::*;
import slave_sequence_item::*;
`include "uvm_macros.svh"

class slave_coverage extends uvm_component;
    `uvm_component_utils(slave_coverage)

    uvm_analysis_export #(slave_seq_item) cov_export;
    uvm_tlm_analysis_fifo #(slave_seq_item) cov_fifo;
    slave_seq_item seq_item_cov;
    covergroup      covcode;
    option.per_instance = 1;
    rx_data_cp: coverpoint seq_item_cov.rx_data[9:8] {
        bins WA = {2'b00};
        bins WD = {2'b01};
        bins RA = {2'b10};
        bins RD = {2'b11};
        illegal_bins others = default;
    }
    ss_cp: coverpoint seq_item_cov.SS_n{
        bins low_len_13 = (1 => 0[*13] => 1);
        bins low_len_23 = (1 => 0[*23] => 1);
        illegal_bins low_too_short = (1 => 0[*1:12] => 1);
        illegal_bins low_mid_wrong = (1 => 0[*14:22] => 1);
        //illegal_bins low_too_long = (1 => 0[*24:$] => 1);
        bins gap_one = (1[*1] => 0);
    }
    cp_mosi_cmd: coverpoint seq_item_cov.MOSI iff (!seq_item_cov.SS_n) {
        bins WA = (0 => 0 => 0); // 000
    }

```

```

    bins WD = (0 == 0 == 1); // 001
    bins RA = (1 == 1 == 0); // 110
    bins RD = (1 == 1 == 1); // 111
}

cross_ssn_mosi: cross ss_cp, rx_data_cp{
    illegal_bins rd_len_wrong = binsof(rx_data_cp.RD) &&
!binsof(ss_cp.low_len_23);
    illegal_bins nonrd_len_wrong =
    (binsof(rx_data_cp.WA) || binsof(rx_data_cp.WD) || binsof(rx_data_cp.RA))
    && !binsof(ss_cp.low_len_13);
}
endgroup

function new(string name="slave_coverage",uvm_component parent=null);
    super.new(name,parent);
    covcode = new();
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    cov_export = new("cov_export",this);
    cov_fifo    = new("cov_fifo",this);
endfunction

function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    cov_export.connect(cov_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
        cov_fifo.get(seq_item_cov);
        covcode.sample();
    end
endtask
endclass
endpackage

```

1.12 Slave Agent

```

package slave_agent_pkg;
import uvm_pkg::*;
import slave_driver_pkg::*;

```

```

import slave_config_pkg::*;
import slave_sequencer_pkg::*;
import slave_monitor_pkg::*;
import slave_sequence_item::*;
`include "uvm_macros.svh"
class slave_agent extends uvm_agent;
`uvm_component_utils(slave_agent)
slave_sequencer sqr;
slave_driver drv;
slave_config slave_cfg;
slave_monitor mon;
uvm_analysis_port #(slave_seq_item) agt_ap;

function new(string name="slave_agent",uvm_component parent=null);
super.new(name,parent);
endfunction

function void build_phase(uvm_phase phase);
super.build_phase(phase);
// pragma coverage off
if(!uvm_config_db#(slave_config)::get(this,"","CFG",slave_cfg)) begin
    `uvm_fatal("build_phase","unable to get configuration object");
end
// pragma coverage on
sqr=slave_sequencer::type_id::create("sqr",this);
drv=slave_driver::type_id::create("drv",this);
mon=slave_monitor::type_id::create("mon",this);
agt_ap=new("agt_ap",this);
endfunction

function void connect_phase(uvm_phase phase);
super.connect_phase(phase);
drv.slave_vif=slave_cfg.slave_vif;
mon.slave_vif=slave_cfg.slave_vif;
drv.seq_item_port.connect(sqr.seq_item_export);
mon.mon_ap.connect(agt_ap);
endfunction

endclass
endpackage

```

1.13 Slave Env

```
package slave_env_pkg;
import uvm_pkg::*;
import slave_agent_pkg::*;
import slave_scoreboard_pkg::*;
import slave_coverage_pkg::*;
`include "uvm_macros.svh"

class slave_env extends uvm_env;
    `uvm_component_utils(slave_env)
    slave_agent agt;
    slave_scoreboard sb;
    slave_coverage cov;
    function new(string name="slave_env",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        agt=slave_agent::type_id::create("agt",this);
        sb=slave_scoreboard::type_id::create("sb",this);
        cov=slave_coverage::type_id::create("cov",this);
    endfunction: build_phase

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        agt.agt_ap.connect(sb.sb_export);
        agt.agt_ap.connect(cov.cov_export);
    endfunction
endclass
endpackage
```

1.14 Slave Test

```
package slave_test_pkg;
import slave_env_pkg::*;
import slave_config_pkg::*;
import uvm_pkg::*;
import slave_seq_pkg::*;
import slave_agent_pkg::*;
import slave_sequencer_pkg::*;
`include "uvm_macros.svh"
```

```

class slave_test extends uvm_test;
  `uvm_component_utils(slave_test)
  slave_env env;
  slave_config slave_cfg;
  virtual slave_if slave_vif;
  slave_main_sequence main_seq;
  slave_reset_sequence reset_seq;
  function new(string name="slave_test",uvm_component parent=null);
    super.new(name,parent);
endfunction
function void build_phase(uvm_phase phase);
  super.build_phase(phase);
  env=slave_env::type_id::create("env",this);
  slave_cfg=slave_config::type_id::create("slave_cfg");
  main_seq=slave_main_sequence::type_id::create("main_seq");
  reset_seq=slave_reset_sequence::type_id::create("reset_seq");
  // pragma coverage off
  if(!uvm_config_db #(virtual slave_if) ::
get(this,"","slave_IF",slave_cfg.slave_vif)) begin
    `uvm_fatal("build_phase","unable to get configuration object");
  end
  // pragma coverage on
  uvm_config_db #(slave_config)::set(this,"*", "CFG",slave_cfg);
endfunction

task run_phase(uvm_phase phase);
  super.run_phase(phase);
  phase.raise_objection(this);
  `uvm_info("run_phase","reset asserted ",UVM_LOW);
  reset_seq.start(env.agt.sqr);
  `uvm_info("run_phase","reset deasserted ",UVM_LOW);
  `uvm_info("run_phase","stimulus generation",UVM_LOW);
  main_seq.start(env.agt.sqr);
  `uvm_info("run_phase","stimulus generation ended",UVM_LOW);
  phase.drop_objection(this);
endtask: run_phase
endclass: slave_test
endpackage

```

1.15 Slave Top

```

import uvm_pkg::*;
`include "uvm_macros.svh"
import slave_test_pkg::*;

```

```

module slave_top();
    bit clk;
    // Clock generator
    initial begin
        clk = 0;
        forever #1 clk = ~clk;
    end

    slave_if slaveif(clk);
    SLAVE dut(
        .clk      (clk),
        .rst_n    (slaveif.rst_n),
        .SS_n     (slaveif.SS_n),
        .MOSI     (slaveif.MOSI),
        .rx_valid (slaveif.rx_valid),
        .MISO     (slaveif.MISO),
        .tx_valid (slaveif.tx_valid),
        .tx_data  (slaveif.tx_data),
        .rx_data  (slaveif.rx_data)
    );
    slave_golden golden(
        .clk      (clk),
        .rst_n    (slaveif.rst_n),
        .SS_n     (slaveif.SS_n),
        .MOSI     (slaveif.MOSI),
        .rx_valid (slaveif.rx_valid_ref),
        .MISO     (slaveif.MISO_ref),
        .tx_valid (slaveif.tx_valid),
        .tx_data  (slaveif.tx_data),
        .rx_data  (slaveif.rx_data_ref)
    );

    initial begin
        uvm_config_db#(virtual
slave_if)::set(null,"uvm_test_top","slave_IF",slaveif);
        run_test("slave_test");
    end
endmodule

```


1.16 Verification Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
SLV_REQ_1	When reset is asserted: cs=IDLE, counter=0, rx_valid=0, received_address=0, MISO idle.	Directed: power-on reset + mid-simulation resets (pulse and long).	cover property p_rst; coverpoint ssn_idle; coverpoint rx_valid_low_on_reset.	Assertions: a_rst_state, a_outputs_cleared. Scoreboard baseline after reset.
SLV_REQ_2	Decode in CHK_CMD: MOSI=0→WRITE; MOSI=1 & !received_address→READ_ADD; MOSI=1 & received_address→READ_DATA.	Constrained-random main sequence + directed one-frame per opcode.	Covergroup cp_cmd on rx_data[9:8] + transition bins; cover property p_chk_cmd_branches.	Assertions: a_chk_cmd_write/ra/rd (ns choice).
SLV_REQ_3	SS_n frame duration: normal 1→0[*13]→1; extended read 1→0[*23]→1.	Directed frames with exact 13 / 23 low cycles; occasional litter tests.	cover property s_normal_frame, s_readext_frame; coverpoint cp_ssn_len.	Assertions: a_ssn_len_normal, a_ssn_len_readext.
SLV_REQ_4	WRITE path: shift 10 bits MSB→LSB into rx_data; rx_valid pulses on last bit (counter==1).	Directed WRITE frames with varied payloads (walking ones/zeros).	Coverpoint last_bit_event; bins for rx_valid pulse; data pattern bins.	Assertions: a_write_lastbit_strobe; Scoreboard compares captured MOSI to rx_data.
SLV_REQ_5	READ_ADDR path: shift 10 bits; when complete set received_address=1 and rx_valid pulses.	Directed READ_ADDR frames; random payloads.	Coverpoint received_address_set; cp_cmd==RA hit; transition RA→CHK_CMD.	Assertions: a_ra_sets_flag; Scoreboard updates address.
SLV_REQ_6	READ_DATA RX subpath (tx_valid=0): capture 8 data bits after header; rx_valid pulses; counter reload=8.	Directed RD with tx_valid=0; randomized data patterns.	Coverpoint tx_valid_off; bins for rx_valid at last bit; payload bins.	Assertions: a_rd_rx_capture; Scoreboard compares rx_data[7:0] with MOSI stream.
SLV_REQ_7	READ_DATA TX subpath (tx_valid=1): drive MISO with tx_data[7:0]; clear MISO at end; clear received_address.	Directed RD with tx_valid=1; try multiple tx_data values.	Coverpoint tx_valid_on; MISO toggle bins; cross(cmd=RD, tx_valid=1).	Assertions: a_miso_drive_seq, a_miso_clear_last, a_clear_received_address.
SLV_REQ_8	Mid-frame abort: SS_n deasserted while active → go to IDLE, clear counter, rx_valid=0, MISO idle.	Directed: deassert SS_n mid-frame in WRITE/RA/RD; also around last bit.	cover property p_abort_to_idle; cp_ssn_edge_midframe.	Assertions: a_abort_returns_idle_in_1cyc; a_outputs_cleared_on_abort.
SLV_REQ_9	Counter safety: never under/overflow; RD clamped ≤8 when tx_valid path; safe indexing for tx_data[counter-1].	Directed: long frames + late drop of tx_valid; random stress.	Cover properties on counter range; coverpoint counter_values.	Assertions: a_counter_bounds; a_safe_indexing_no_neg1.
SLV_REQ_10	FSM: reach all states (IDLE, CHK_CMD, WRITE, READ_ADD, READ_DATA) & all legal transitions; illegal default unreachable.	Combination of directed frames and random mix; include self-loops.	FSM coverage report (states & transitions).	Assertions: a_state_enum_valid; Waiver/coverage_off for default if provably unreachable.
SLV_REQ_11	Idle behavior: when SS_n=1, MISO is tri-stated/idle per spec; no spurious rx_valid.	Insert 1-2 cycle gaps between frames; random idle stretches.	Coverpoint miso_idle_level; coverpoint rx_valid_idle=0.	Assertions: a_idle_miso, a_no_rxvalid_in_idle.
SLV_REQ_12	Data/address field correctness across operations: write/read consistency, scoreboard agreement.	Randomized sequences with directed corner cases (000/FFF, walking patterns).	Coverpoints on data patterns; crosses (cmd x tx_valid); ignore_bins for illegal combos.	Scoreboard end-to-end comparisons; Assertions on protocol legality.

1.17 Bug Report

- **Missing Default Case:** Initial lack of default: ns = IDLE in next state logic could cause undefined behavior (fixed, but verify coverage).
- **Counter Reset Issue:** counter not initialized on reset initially (now fixed, confirm synthesis behavior).
- **Counter Bug in READ_DATA:** Reassigns counter ≤ 9 when it should be 0, risking an infinite loop.
- **MISO Handling:** Undefined when tx_valid is low; add default MISO ≤ 0.
- **Transaction Length:** 10-bit rx_data with 11-cycle counter mismatch; adjust to 10 or 11 bits.
- **State Exit:** No exit from WRITE, READ_ADD, or READ_DATA when counter == 0; add condition.

1.18 Assertion Table

Assertion Name	Description
a_reset_outputs_low	Ensures MISO, rx_valid, and rx_data are 0 when reset is active
c_reset_outputs_low	Covers the scenario where MISO, rx_valid, and rx_data are 0 during reset
a_counter_load_in_chk	Ensures counter is loaded with 10 when entering CHK_CMD with SS_n low
c_counter_load_in_chk	Covers the counter load to 10 in CHK_CMD with SS_n low
c_ssn_rxvalid_write	Covers rx_valid assertion after 10 cycles and SS_n rising within 12 cycles for WRITE or READ_ADD
a_ssn_rxvalid_read	Asserts rx_valid after 10 cycles and SS_n rising within 23 cycles for READ_DATA without tx_valid

c_ssn_rxvalid_read	Covers rx_valid assertion after 10 cycles and SS_n rising within 23 cycles for READ_DATA without tx_valid
prop_idle_to_chk	Ensures transition from IDLE to CHK_CMD when SS_n falls
prop_idle_hold	Ensures IDLE state holds when SS_n remains high
prop_chk_to_idle	Ensures transition from CHK_CMD to IDLE when SS_n rises
prop_chk_to_read_add	Ensures transition from CHK_CMD to READ_ADD for read address when MOSI is 1 and no address received
prop_chk_to_read_data	Ensures transition from CHK_CMD to READ_DATA for read data when MOSI is 1 and address received
prop_write_hold	Ensures WRITE state holds while SS_n remains low
prop_write_to_idle	Ensures transition from WRITE to IDLE when SS_n rises
prop_readadd_hold	Ensures READ_ADD state holds while SS_n remains low
prop_readadd_to_idle	Ensures transition from READ_ADD to IDLE when SS_n rises
prop_readdata_hold	Ensures READ_DATA state holds while SS_n remains low
prop_readdata_to_idle	Ensures transition from READ_DATA to IDLE when SS_n rises

1.19 Coverage Report

Coverage Report by instance with details

```
=====
=== Instance: /slave_top/slaveif
=== Design Unit: work.slave.if
=====
Toggle Coverage:
-----
Enabled Coverage      Bins    Hits    Misses  Coverage
-----
Toggles              74      74      0    100.00%

=====Toggle Details=====
Toggle Coverage for instance /slave_top/slaveif --
Node      1H->0L      0L->1H  "Coverage"
-----
MISO      1            1    100.00
MISO_ref  1            1    100.00
MOSI      1            1    100.00
SS_0      1            1    100.00
clk       1            1    100.00
rst_n     1            1    100.00
rx_data[9-0] 1            1    100.00
rx_data_ref[9-0] 1            1    100.00
rx_valid  1            1    100.00
rx_valid_ref 1            1    100.00
tx_data[7-0] 1            1    100.00
tx_valid  1            1    100.00

Total Node Count   =    37
Toggled Node Count =    37
Untoggled Node Count =    0

Toggle Coverage    =   100.00% (74 of 74 bins)
```

```
=====
=== Instance: /slave_top/dut
=== Design Unit: work.SLAVE
=====
Assertion Coverage:
-----
Assertions          15      15      0    100.00%
Name                File(Line)      Failure Count    Pass Count
-----
/slave_top/dut/assert _pcop_readdata_to_idle
SPI_slave.v(264)      0      1
/slave_top/dut/assert _pcop_readdata_hold
SPI_slave.v(255)      0      1
/slave_top/dut/assert _pcop_readadd_to_idle
SPI_slave.v(246)      0      1
/slave_top/dut/assert _pcop_readadd_hold
SPI_slave.v(237)      0      1
/slave_top/dut/assert _pcop_write_to_idle
SPI_slave.v(228)      0      1
/slave_top/dut/assert _pcop_write_hold
SPI_slave.v(219)      0      1
/slave_top/dut/assert _pcop_chk_to_read_data
SPI_slave.v(210)      0      1
/slave_top/dut/assert _pcop_chk_to_read_add
SPI_slave.v(200)      0      1
/slave_top/dut/assert _pcop_chk_to_write
SPI_slave.v(190)      0      1
/slave_top/dut/assert _pcop_chk_to_idle
SPI_slave.v(181)      0      1
/slave_top/dut/assert _pcop_idle_hold
SPI_slave.v(172)      0      1
/slave_top/dut/assert _pcop_idle_to_chk
SPI_slave.v(163)      0      1
/slave_top/dut/a_counter_load_in_chk
SPI_slave.v(137)      0      1
/slave_top/dut/a_ssn_rxvalid_write
SPI_slave.v(143)      0      1
/slave_top/dut/a_ssn_rxvalid_read
SPI_slave.v(151)      0      1

Branch Coverage:
-----
Enabled Coverage      Bins    Hits    Misses  Coverage
-----
Branches              34      34      0    100.00%

=====Branch Details=====
```

```

Branches          34      34      0 100.00%

=====Branch Details=====
Branch Coverage for instance /slave_top/dut
File SPI_slave.v
-----IF Branch-----
Line      Item      Count      Source
-----
17              414      Count coming in to IF
17          1          8      if (~CSL_n) begin
20          1         406      else begin
Branch totals: 2 hits of 2 branches = 100.00%

-----CASE Branch-----
Line      Item      Count      Source
-----
26              1143      Count coming in to CASE
27          1          200      IDLE : begin
33          1          101      CHK_CMD : begin
47          1          332      WRITE : begin
53          1          252      READ_ADD : begin
59          1          257      READ_DATA : begin
66          1           1      default: ns = IDLE;
Branch totals: 6 hits of 6 branches = 100.00%

-----IF Branch-----
Line      Item      Count      Source
-----
28              200      Count coming in to IF
28          1          100      if (SS_n)
30          1          100      else
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
Line      Item      Count      Source
-----
34              101      Count coming in to IF
34          1           1      if (SS_0)
36          1          100      else begin
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
Line      Item      Count      Source
-----
37              100      Count coming in to IF
37          1           49      if (~MOSI)
39          1           51      else begin
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
Line      Item      Count      Source
-----
40              51      Count coming in to IF
40          1           25      if (!received_address) //when equal 0 not 1
42          1           26      else
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
Line      Item      Count      Source
-----
48              332      Count coming in to IF
48          1           49      if (SS_0)
50          1          283      else
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
Line      Item      Count      Source
-----
54              252      Count coming in to IF
54          1           25      if (SS_0)
56          1          227      else
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
Line      Item      Count      Source
-----
60              257      Count coming in to IF
60          1           24      if (SS_n)
62          1          233      else
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
Line      Item      Count      Source
-----
71              1515      Count coming in to IF
71          1           8      if (~CSL_n) begin
78          1          1507      else begin
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
Line      Item      Count      Source
-----
87              588      Count coming in to IF
87          1          490      if (counter > 0) begin
90          1           98      end else
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
Line      Item      Count      Source
-----
94              331      Count coming in to IF
94          1          250      if (counter > 0) begin
97          1           81      end else begin
Branch totals: 2 hits of 2 branches = 100.00%

```

```

-----IF Branch-----
94          1          331    Count coming in to IF
94          1          250    if (counter > 0) begin
97          1          81      end else begin
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
103         1          379    Count coming in to IF
103         1          120    if (tx_valid) begin
113         1          259    else begin
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
105         1          120    Count coming in to IF
105         1          100    if (counter > 0) begin
109         1          20     end else begin
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
114         1          259    Count coming in to IF
114         1          228    if (counter > 0) begin
117         1          31     end else begin
Branch totals: 2 hits of 2 branches = 100.00%

Condition Coverage:
Enabled Coverage      Bins  Covered  Misses  Coverage
-----
Conditions            4      4        0    100.00%

=====Condition Details=====

Condition Coverage for instance /slave_top/dut --

File SPI_slave.v
-----Focused Condition View-----
Line      87 Item      1 (counter > 0)
Condition totals: 1 of 1 input term covered = 100.00%

Input Term  Covered  Reason for no coverage  Hint
-----
(counter > 0)      Y

Rows:      Hits  FEC Target      Non-masking condition(s)
-----
Row 1:      1    (counter > 0)_0      -
Row 2:      1    (counter > 0)_1      -

-----Focused Condition View-----
Line      94 Item      1 (counter > 0)
Condition totals: 1 of 1 input term covered = 100.00%

Input Term  Covered  Reason for no coverage  Hint
-----
(counter > 0)      Y

Rows:      Hits  FEC Target      Non-masking condition(s)
-----
Row 1:      1    (counter > 0)_0      -
Row 2:      1    (counter > 0)_1      -

-----Focused Condition View-----
Line     105 Item      1 (counter > 0)
Condition totals: 1 of 1 input term covered = 100.00%

Input Term  Covered  Reason for no coverage  Hint
-----
(counter > 0)      Y

Rows:      Hits  FEC Target      Non-masking condition(s)
-----
Row 1:      1    (counter > 0)_0      -
Row 2:      1    (counter > 0)_1      -

-----Focused Condition View-----
Line     114 Item      1 (counter > 0)
Condition totals: 1 of 1 input term covered = 100.00%

Input Term  Covered  Reason for no coverage  Hint
-----
(counter > 0)      Y

Rows:      Hits  FEC Target      Non-masking condition(s)
-----

```

Ln 1, Col 1 | 185,373 characters

Plain text

Line 114 Item 1 (counter > 0)
Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
(counter > 0)	Y		
Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(counter > 0)_0	-
Row 2:	1	(counter > 0)_1	-

Directive Coverage:
Directives 16 16 0 100.00%

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status
/slave_top/dut/cover__prop_readdata_to_idle	SLAVE	Verilog	SVA	SPI_slave.v(266)	24	Covered
/slave_top/dut/cover__prop_readdata_hold	SLAVE	Verilog	SVA	SPI_slave.v(257)	444	Covered
/slave_top/dut/cover__prop_readadd_to_idle	SLAVE	Verilog	SVA	SPI_slave.v(248)	25	Covered
/slave_top/dut/cover__prop_readadd_hold	SLAVE	Verilog	SVA	SPI_slave.v(239)	395	Covered
/slave_top/dut/cover__prop_write_to_idle	SLAVE	Verilog	SVA	SPI_slave.v(230)	49	Covered
/slave_top/dut/cover__prop_write_hold	SLAVE	Verilog	SVA	SPI_slave.v(221)	539	Covered
/slave_top/dut/cover__prop_chk_to_read_data	SLAVE	Verilog	SVA	SPI_slave.v(212)	25	Covered
/slave_top/dut/cover__prop_chk_to_read_add	SLAVE	Verilog	SVA	SPI_slave.v(202)	25	Covered
/slave_top/dut/cover__prop_chk_to_write	SLAVE	Verilog	SVA	SPI_slave.v(192)	49	Covered
/slave_top/dut/cover__prop_chk_to_idle	SLAVE	Verilog	SVA	SPI_slave.v(183)	1	Covered
/slave_top/dut/cover__prop_idle_hold	SLAVE	Verilog	SVA	SPI_slave.v(174)	3	Covered
/slave_top/dut/cover__prop_idle_to_chk	SLAVE	Verilog	SVA	SPI_slave.v(165)	100	Covered
/slave_top/dut/c_reset_outputs_low	SLAVE	Verilog	SVA	SPI_slave.v(133)	8	Covered
/slave_top/dut/c_counter_load_in_chk	SLAVE	Verilog	SVA	SPI_slave.v(139)	99	Covered
/slave_top/dut/c_ssn_rxvalid_write	SLAVE	Verilog	SVA	SPI_slave.v(147)	74	Covered
/slave_top/dut/c_ssn_rxvalid_read	SLAVE	Verilog	SVA	SPI_slave.v(155)	15	Covered

FSM Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
FSM States	5	5	0	100.00%
FSM Transitions	8	8	0	100.00%

=====FSM Details=====

FSM Coverage for instance /slave_top/dut --

FSM_ID: cs

Current State Object : cs

State Value MapInfo :

Line	State Name	Value
27	IDLE	0
33	CHK_CMD	2
59	READ_DATA	4
53	READ_ADD	3
47	WRITE	1

Covered States :

State	Hit count
IDLE	116
CHK_CMD	100
READ_DATA	50
READ_ADD	50
WRITE	98

Covered Transitions :

Line	Trans_ID	Hit count	Transition
31	0	100	IDLE -> CHK_CMD
43	1	25	CHK_CMD -> READ_DATA
41	2	25	CHK_CMD -> READ_ADD
38	3	49	CHK_CMD -> WRITE
35	4	1	CHK_CMD -> IDLE
61	5	24	READ_DATA -> IDLE
55	6	25	READ_ADD -> IDLE
49	7	49	WRITE -> IDLE

Summary	Bins	Hits	Misses	Coverage
FSM States	5	5	0	100.00%
FSM Transitions	8	8	0	100.00%
Statement Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage
Statements	41	41	0	100.00%

```

Toggle Coverage:
Enabled Coverage
-----
Toggles          Bins    Hits    Misses  Coverage
-----
                  72      72      0      100.00%

```

```

=====Toggle Details=====

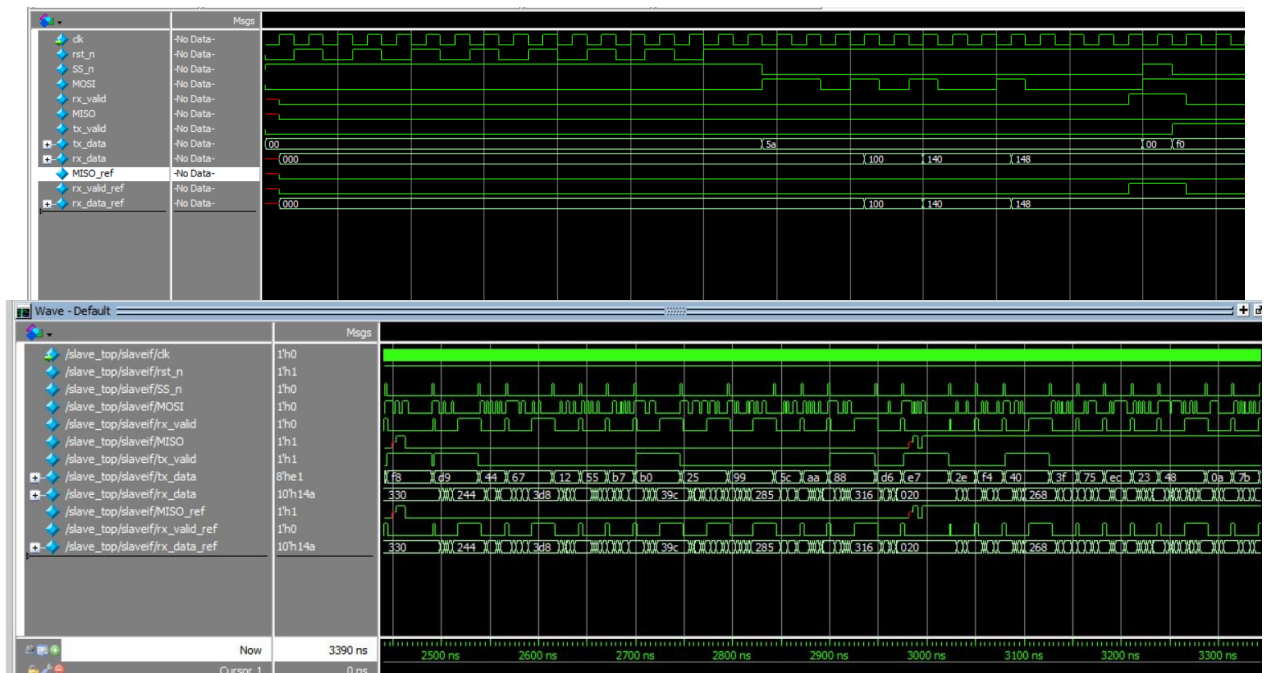
```

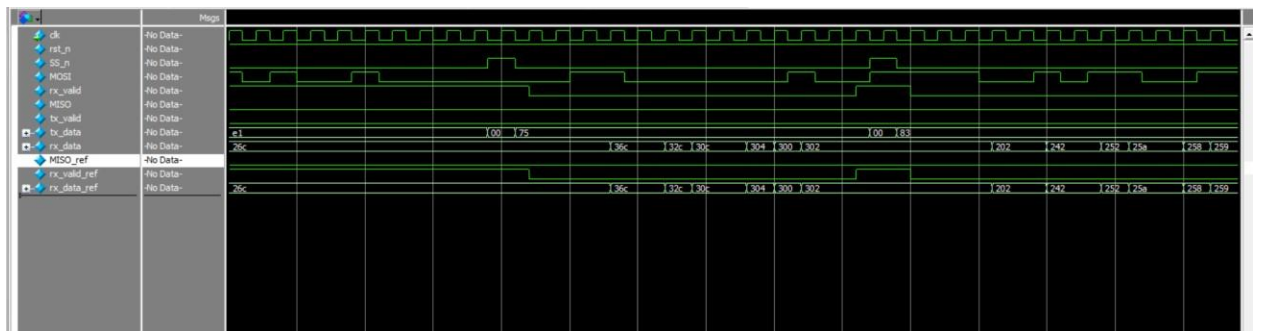
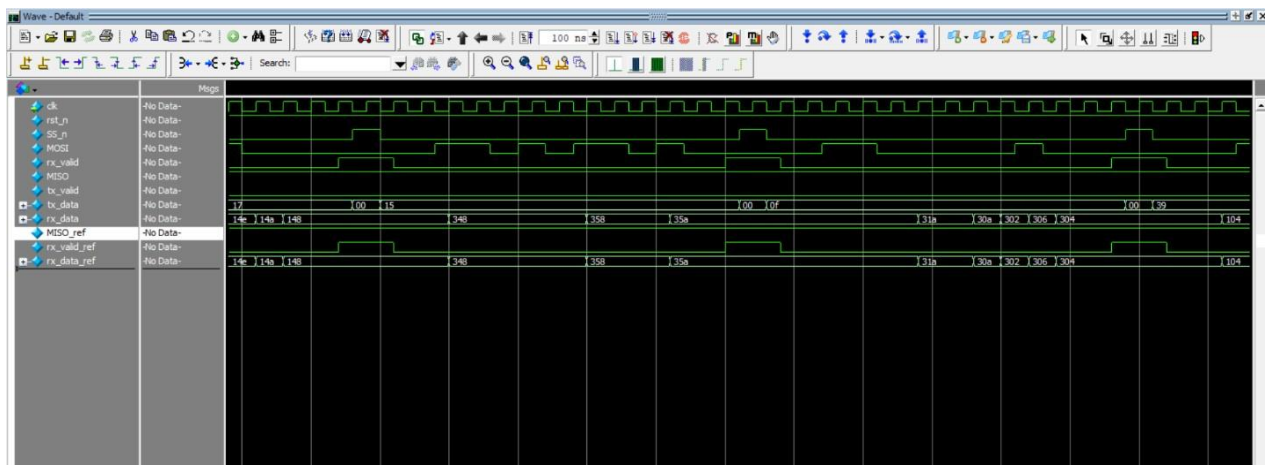
1.20 Questasim

```

UVM_INFO slave_test.sv(43) @ 3390: uvm_test_top [run_phase] stimulus generation ended
UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 3390: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
UVM_INFO slave_scoreboard.sv(49) @ 3390: uvm_test_top.env.sv [report_phase] total successful transactions:1695
UVM_INFO slave_scoreboard.sv(50) @ 3390: uvm_test_top.env.sv [report_phase] total failed transactions:0
--- UVM Report Summary ---
** Report counts by severity
UVM_INFO : 11
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0
** Report counts by id
[Questa UVM] 2
[RNIST] 1
[RST_SEQ] 1
[TEST_DONE] 1
[report_phase] 2
[run_phase] 4
** Note: $finish : D:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)

```





Cover Directives													
Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Pea
/slave_top/dut/cov...	SVA	✓	Off	24	1	Unli...	1	100%	✓	✓	0	0	
/slave_top/dut/cov...	SVA	✓	Off	444	1	Unli...	1	100%	✓	✓	0	0	
/slave_top/dut/cov...	SVA	✓	Off	25	1	Unli...	1	100%	✓	✓	0	0	
/slave_top/dut/cov...	SVA	✓	Off	395	1	Unli...	1	100%	✓	✓	0	0	
/slave_top/dut/cov...	SVA	✓	Off	49	1	Unli...	1	100%	✓	✓	0	0	
/slave_top/dut/cov...	SVA	✓	Off	539	1	Unli...	1	100%	✓	✓	0	0	
/slave_top/dut/cov...	SVA	✓	Off	25	1	Unli...	1	100%	✓	✓	0	0	
/slave_top/dut/cov...	SVA	✓	Off	25	1	Unli...	1	100%	✓	✓	0	0	
/slave_top/dut/cov...	SVA	✓	Off	49	1	Unli...	1	100%	✓	✓	0	0	
/slave_top/dut/cov...	SVA	✓	Off	1	1	Unli...	1	100%	✓	✓	0	0	
/slave_top/dut/cov...	SVA	✓	Off	3	1	Unli...	1	100%	✓	✓	0	0	
/slave_top/dut/cov...	SVA	✓	Off	100	1	Unli...	1	100%	✓	✓	0	0	
/slave_top/dut/c_r...	SVA	✓	Off	8	1	Unli...	1	100%	✓	✓	0	0	
/slave_top/dut/c_c...	SVA	✓	Off	99	1	Unli...	1	100%	✓	✓	0	0	
/slave_top/dut/c_s...	SVA	✓	Off	74	1	Unli...	1	100%	✓	✓	0	0	
/slave_top/dut/c_s...	SVA	✓	Off	15	1	Unli...	1	100%	✓	✓	0	0	


```

    logic [7:0] dout_ref;
    logic      tx_valid_ref;
endinterface
endpackage

```

2.2 SVA module

```

module ram_sva (
    input logic clk,
    input logic rst_n,
    input logic [9:0] din,
    input logic rx_valid,
    input logic [7:0] dout,
    input logic tx_valid
);

    // 1. Reset clears outputs
    property reset_clear;
        @(posedge clk)
            (!rst_n) | => (dout == 0 && tx_valid == 0);
    endproperty

    // 2. During WA/WD/RA, tx_valid must stay low
    property tx_valid_off;
        @(posedge clk) disable iff (!rst_n)
            (rx_valid && (din[9:8] inside {2'b00,2'b01,2'b10})) | => !tx_valid;
    endproperty

    // 3. After RD, tx_valid must rise then eventually fall
    property tx_valid_on;
        @(posedge clk) disable iff (!rst_n)
            (rx_valid && din[9:8]==2'b11) | => (tx_valid ##1 !tx_valid[->1]);
    endproperty

    // 4. WA must eventually be followed by WD
    property write_operation;
        @(posedge clk) disable iff (!rst_n)
            (rx_valid && din[9:8]==2'b00) | -> ##[1:$] (rx_valid && din[9:8]==2'b01);
    endproperty

    // 5. RA must eventually be followed by RD
    property read_operation;
        @(posedge clk) disable iff (!rst_n)
            (rx_valid && din[9:8]==2'b10) | -> ##[1:$] (rx_valid && din[9:8]==2'b11);
    endproperty

```

```

assert property (reset_clear);
assert property (tx_valid_off);
assert property (tx_valid_on);
assert property (write_operation);
assert property (read_operation);

cover property (reset_clear);
cover property (tx_valid_off);
cover property (tx_valid_on);
cover property (write_operation);
cover property (read_operation);

endmodule

```

2.3 Sequence item

```

package ram_sequence_item;
import uvm_pkg::*;
`include "uvm_macros.svh"
class ram_seq_item extends uvm_sequence_item;
`uvm_object_utils(ram_seq_item)
rand bit clk;
rand bit [9:0] din;
rand bit rst_n;
rand bit rx_valid;
rand bit [7:0] dout;
rand bit tx_valid;
rand bit [7:0] dout_ref;
rand bit tx_valid_ref;
    constraint rst_c {rst_n dist {1:/90, 0:/10}};

    constraint rx_valid_c {rx_valid dist {1:/80, 0:/20}};
    constraint din_c {
        if (rx_valid) {
            din[9:8] dist {2'b11:/50, 2'b00:/20, 2'b01:/20, 2'b10:/10}; // 50% read
data
        }
    }

function new(string name="ram_seq_item");
super.new(name);
endfunction

function string convert2string();

```

```

return $sformatf("%s rst=0b%0b din=0b%0b,rx valid=0b%0b,dout=%0s,tx
valid=0b%0b,dout_ref=%0s,tx
valid_ref=0b%0b",super.convert2string(),rst_n,din,rx_valid,dout,tx_valid,dout_ref
,tx_valid_ref);
endfunction

function string convert2string_stimulus();
    return $sformatf(" rst=0b%0b din=0b%0b,rx valid=0b%0b",rst_n,din,rx_valid);
endfunction

endclass
endpackage

```

2.4 Sequence

```

package ram_seq_pkg;
import uvm_pkg::*;
import ram_sequence_item::*;
`include "uvm_macros.svh"
class ram_reset_sequence extends uvm_sequence #(ram_seq_item);
    `uvm_object_utils(ram_reset_sequence)
    ram_seq_item seq_item;

    function new(string name="ram_reset_sequence");
        super.new(name);
    endfunction

    task body;
        seq_item=ram_seq_item::type_id::create("seq_item");
        start_item(seq_item);
        seq_item.rst_n=0;
        seq_item.din=0;
        seq_item.rx_valid=0;
        finish_item(seq_item);
    endtask
endclass

class ram_write_only_sequence extends uvm_sequence #(ram_seq_item);
    `uvm_object_utils(ram_write_only_sequence)
    ram_seq_item seq_item;
    bit [1:0] last_op;

    function new(string name="ram_write_only_sequence");
        super.new(name);
    endfunction
endclass

```

```

endfunction

task body();
repeat (50) begin
    seq_item = ram_seq_item::type_id::create("seq_item");
    start_item(seq_item);

    assert(seq_item.randomize() with {
        if (last_op == 2'b00 || last_op == 2'b01)
            din[9:8] inside {2'b00, 2'b01}; // WA or WD
        else
            din[9:8] == 2'b00; // First must be WA
        rx_valid == 1;
    });

    finish_item(seq_item);
    last_op = seq_item.din[9:8];
end
endtask

function void post_randomize();
    if (last_op == 2'b00 || last_op == 2'b01)
        assert(seq_item.din[9:8] inside {2'b00, 2'b01})
        else `uvm_error("WRITE_ONLY_SEQ", "Ordering violation: WA/WD must be
followed by WA or WD");
    endfunction
endclass

class ram_read_only_sequence extends uvm_sequence #(ram_seq_item);
    `uvm_object_utils(ram_read_only_sequence)
    ram_seq_item seq_item;
    bit [1:0] last_op;

    function new(string name="ram_read_only_sequence");
        super.new(name);
    endfunction

task body();
repeat (50) begin
    seq_item = ram_seq_item::type_id::create("seq_item");
    start_item(seq_item);

    assert(seq_item.randomize() with {
        if (last_op == 2'b10) // RA -> must be RD
            din[9:8] == 2'b11;
    });
end
endtask

```

```

        else if (last_op == 2'b11)      // RD -> must be RA
            din[9:8] == 2'b10;
        else                            // first operation
            din[9:8] == 2'b10;          // start with RA
        rx_valid == 1;
    });

    finish_item(seq_item);
    last_op = seq_item.din[9:8];
end
endtask

function void post_randomize();

    case (last_op)
        2'b10: assert(seq_item.din[9:8] == 2'b11)
                else `uvm_error("READ_ONLY_SEQ", "After RA must be RD")
        2'b11: assert(seq_item.din[9:8] == 2'b10)
                else `uvm_error("READ_ONLY_SEQ", "After RD must be RA")
    endcase
endfunction
endclass

class ram_write_read_sequence extends uvm_sequence #(ram_seq_item);
    `uvm_object_utils(ram_write_read_sequence)
    ram_seq_item seq_item;
    bit [1:0] last_op;

    function new(string name="ram_write_read_sequence");
        super.new(name);
    endfunction

    task body();
        repeat (1000) begin
            seq_item = ram_seq_item::type_id::create("seq_item");
            start_item(seq_item);

            case (last_op)
                2'b00: assert(seq_item.randomize() with { // WA -> WA or WD
                    din[9:8] inside {2'b00, 2'b01}; rx_valid == 1;
                });

                2'b01: assert(seq_item.randomize() with { // WD -> 60% RA, 40% WA
                    rx_valid == 1;
                    din[9:8] dist {2'b10:/60, 2'b00:/40};
                });
            endcase
        end
    endtask
endclass

```

```

    });

    2'b10: assert(seq_item.randomize() with { // RA -> RA or RD
        din[9:8] inside {2'b10, 2'b11}; rx_valid == 1;
    });

    2'b11: assert(seq_item.randomize() with { // RD -> 60% WA, 40% RA
        rx_valid == 1;
        din[9:8] dist {2'b00:/60, 2'b10:/40};
    });

    default: assert(seq_item.randomize() with { // First op must be WA
        din[9:8] == 2'b00; rx_valid == 1;
    });
endcase

finish_item(seq_item);
last_op = seq_item.din[9:8];
end
endtask

function void post_randomize();
case (last_op)
    2'b00: assert(seq_item.din[9:8] inside {2'b00, 2'b01})
        else begin
            `uvm_error("WR_SEQ", "WA must be followed by WA or WD")
        end
    2'b01: assert(seq_item.din[9:8] inside {2'b00, 2'b10})
        else begin
            `uvm_error("WR_SEQ", "WD must be followed by WA or RA")
        end
    2'b10: assert(seq_item.din[9:8] inside {2'b10, 2'b11})
        else begin
            `uvm_error("WR_SEQ", "RA must be followed by RA or RD")
        end
    2'b11: assert(seq_item.din[9:8] inside {2'b00, 2'b10})
        else begin
            `uvm_error("WR_SEQ", "RD must be followed by WA or RA")
        end
endcase
endfunction
endclass

class ram_main_sequence extends uvm_sequence #(ram_seq_item);
    `uvm_object_utils(ram_main_sequence)

```

```

    ram_seq_item seq_item;

    function new(string name="ram_main_sequence");
        super.new(name);
    endfunction

    task body();
        repeat (15000) begin
            seq_item = ram_seq_item::type_id::create("seq_item");
            start_item(seq_item);
            assert(seq_item.randomize());
            finish_item(seq_item);
        end
    endtask
endclass

endpackage

```

2.5 Sequencer

```

package ram_sequencer_pkg;
import uvm_pkg::*;
import ram_sequence_item::*;
`include "uvm_macros.svh"
class ram_sequencer extends uvm_sequencer #(ram_seq_item);
    `uvm_component_utils(ram_sequencer)
    function new(string name="ram_sequencer",uvm_component parent =null);
        super.new(name,parent);
    endfunction
endclass
endpackage

```

2.6 Configuration Object

```

package ram_config_pkg ;
import uvm_pkg::*;
`include "uvm_macros.svh"

class ram_config extends uvm_object;
    `uvm_object_utils(ram_config)
    virtual ram_if ram_vif;

```



```

function new (string name="ram_config");
    super.new(name);
endfunction
endclass
endpackage

```

2.7 Driver

```

package ram_driver_pkg;
import uvm_pkg::*;
import ram_sequence_item::*;
`include "uvm_macros.svh"
class ram_driver extends uvm_driver #(ram_seq_item);
    `uvm_component_utils(ram_driver)
    virtual ram_if ram_vif;
    ram_seq_item stim_seq_item;
    function new(string name="ram_driver",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);

        forever begin
            stim_seq_item=ram_seq_item::type_id::create("stim_seq_item");
            seq_item_port.get_next_item(stim_seq_item);
            ram_vif.din=stim_seq_item.din;
            ram_vif.rst_n=stim_seq_item.rst_n;
            ram_vif.rx_valid=stim_seq_item.rx_valid;
            @(negedge ram_vif.clk);
            seq_item_port.item_done();
            `uvm_info("run_phase",stim_seq_item.convert2string_stimulus(),UVM_HIGH)
        end
    endtask
endclass
endpackage

```

2.8 Monitor

```

package ram_monitor_pkg;
import uvm_pkg::*;
import ram_sequence_item::*;
`include "uvm_macros.svh"
class ram_monitor extends uvm_monitor;

```

```

`uvm_component_utils(ram_monitor)
virtual ram_if ram_vif;
ram_seq_item rsp_seq_item;
uvm_analysis_port #(ram_seq_item) mon_ap;
function new(string name="ram_monitor",uvm_component parent=null);
super.new(name,parent);
endfunction
function void build_phase(uvm_phase phase);
super.build_phase(phase);
mon_ap=new("mon_ap",this);
endfunction

task run_phase(uvm_phase phase);
super.run_phase(phase);

forever begin
rsp_seq_item=ram_seq_item::type_id::create("rsp_seq_item");
@(negedge ram_vif.clk);
rsp_seq_item.din = ram_vif.din;
rsp_seq_item.rx_valid = ram_vif.rx_valid;
rsp_seq_item.rst_n = ram_vif.rst_n;
rsp_seq_item.dout = ram_vif.dout;
rsp_seq_item.tx_valid = ram_vif.tx_valid;
rsp_seq_item.dout_ref = ram_vif.dout_ref;
rsp_seq_item.tx_valid_ref = ram_vif.tx_valid_ref;
mon_ap.write(rsp_seq_item);
`uvm_info("run_phase",rsp_seq_item.convert2string(),UVM_HIGH)
end
endtask
endclass
endpackage

```

2.9 Scoreboard

```

package ram_scoreboard_pkg;
import uvm_pkg::*;
import ram_sequence_item::*;
`include "uvm_macros.svh"

class ram_scoreboard extends uvm_scoreboard;
`uvm_component_utils(ram_scoreboard)
uvm_analysis_export #(ram_seq_item) sb_export;
uvm_tlm_analysis_fifo #(ram_seq_item) sb_fifo;
ram_seq_item seq_item_sb;

```

```

integer error_count=0;
integer correct_count=0;
virtual ram_if vif;

function new(string name="ram_scoreboard",uvm_component parent=null);
super.new(name,parent);
endfunction

function void build_phase(uvm_phase phase);
super.build_phase(phase);
sb_export=new("sb_export",this);
sb_fifo=new("sb_fifo",this);
endfunction

function void connect_phase(uvm_phase phase);
super.connect_phase(phase);
sb_export.connect(sb_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);
super.run_phase(phase);
forever begin
    sb_fifo.get(seq_item_sb);
    if (seq_item_sb.dout != seq_item_sb.dout_ref || seq_item_sb.tx_valid !=
seq_item_sb.tx_valid_ref) begin
        `uvm_error("run_phase",$sformatf("Mismatch: %s | DUT dout=%0h
tx_valid=%0b | REF dout=%0h tx_valid=%0b",
            seq_item_sb.convert2string(), seq_item_sb.dout,
seq_item_sb.tx_valid, seq_item_sb.dout_ref, seq_item_sb.tx_valid_ref));
        error_count++;
    end
    else begin
        `uvm_info("run_phase",$sformatf("correct
out:%s",seq_item_sb.convert2string()),UVM_HIGH);
        correct_count++;
    end
end
endtask

function void report_phase(uvm_phase phase);
super.report_phase(phase);
`uvm_info("report_phase",$sformatf("total successful
transactions:%0d",correct_count),UVM_MEDIUM);
`uvm_info("report_phase",$sformatf("total failed
transactions:%0d",error_count),UVM_MEDIUM);

```

```

endfunction

endclass

endpackage

```

2.10 Coverage

```

package ram_coverage_pkg;
import uvm_pkg::*;
import ram_sequence_item::*;
`include "uvm_macros.svh"

class ram_coverage extends uvm_component;
    `uvm_component_utils(ram_coverage)

    uvm_analysis_export #(ram_seq_item) cov_export;
    uvm_tlm_analysis_fifo #(ram_seq_item) cov_fifo;
    ram_seq_item seq_item_cov;

    covergroup covcode;
        din_cp: coverpoint seq_item_cov.din[9:8]{
            bins wr_addr = {2'b00};
            bins wr_data = {2'b01};
            bins rd_addr = {2'b10};
            bins rd_data = {2'b11};
        }

        //check write data after write address
        wr_data_cp: coverpoint seq_item_cov.din[7:0]
        iff(seq_item_cov.din[9:8]==2'b01) {
            bins wr_data_0 = {8'h00};
            bins wr_data_max = {8'hFF};
            bins wr_data_min = {8'h01};
            bins wr_data_default = default;
        }
        //check read data after read address
        rd_data_cp: coverpoint seq_item_cov.din[7:0]
        iff(seq_item_cov.din[9:8]==2'b11) {
            bins rd_data_0 = {8'h00};
            bins rd_data_max = {8'hFF};
            bins rd_data_min = {8'h01};
            bins rd_data_default = default;
        }
        //check write address => write data => read address => read data
        trans_cp: coverpoint seq_item_cov.din[9:8] {

```

```

        bins wraddr_to_wrdata = (2'b00 => 2'b01);
        bins wrdata_to_rdaddr = (2'b01 => 2'b10);
        bins rdaddr_to_rddata = (2'b10 => 2'b11);
        bins full_sequence = (2'b00 => 2'b01 => 2'b10 => 2'b11);
    }
    rx_valid_cp: coverpoint seq_item_cov.rx_valid {
        bins low = {1'b0};
        bins high = {1'b1};
    }

    tx_valid_cp: coverpoint seq_item_cov.tx_valid {
        bins low = {1'b0};
        bins high = {1'b1};
    }
    cross_din_rxvalid: cross din_cp, rx_valid_cp {
        bins active_cross = binsof(din_cp) && binsof(rx_valid_cp.high) ;
    }

    cross_rddata_txvalid: cross din_cp, tx_valid_cp {
        bins valid_read = binsof(din_cp.rd_data) && binsof(tx_valid_cp.high);
        illegal_bins ra_high = binsof(din_cp.rd_addr) &&
binsof(tx_valid_cp.high);
        illegal_bins wr_high = binsof(din_cp.wr_addr) &&
binsof(tx_valid_cp.high);
        illegal_bins wd_high = binsof(din_cp.wr_data) &&
binsof(tx_valid_cp.high);

    }

endgroup

function new(string name="ram_coverage", uvm_component parent=null);
    super.new(name, parent);
    covcode = new();
endfunction

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    cov_export = new("cov_export", this);
    cov_fifo = new("cov_fifo", this);
endfunction

function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);

```

```

        cov_export.connect(cov_fifo.analysis_export);
    endfunction

    task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            cov_fifo.get(seq_item_cov);
            covcode.sample();
        end
    endtask
endclass
endpackage

```

2.11 Agent

```

package ram_agent_pkg;
import uvm_pkg::*;
import ram_driver_pkg::*;
import ram_config_pkg::*;
import ram_sequencer_pkg::*;
import ram_monitor_pkg::*;
import ram_sequence_item::*;
`include "uvm_macros.svh"
class ram_agent extends uvm_agent;
    `uvm_component_utils(ram_agent)
    ram_sequencer sqr;
    ram_driver drv;
    ram_config ram_cfg;
    ram_monitor mon;
    uvm_analysis_port #(ram_seq_item) agt_ap;

    function new(string name="ram_agent",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if(!uvm_config_db #(ram_config)::get(this,"","CFG",ram_cfg)) begin
            `uvm_fatal("build_phase","unable to get configuration object");
        end
        sqr=ram_sequencer::type_id::create("sqr",this);
        drv=ram_driver::type_id::create("drv",this);
    endfunction
endclass

```

```

    mon=ram_monitor::type_id::create("mon",this);
    agt_ap=new("agt_ap",this);
endfunction

function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    drv.ram_vif=ram_cfg.ram_vif;
    mon.ram_vif=ram_cfg.ram_vif;
    drv.seq_item_port.connect(sqr.seq_item_export);
    mon.mon_ap.connect(agt_ap);
endfunction

endclass
endpackage

```

2.12 ENV package

```

package ram_env_pkg;
import uvm_pkg::*;
import ram_agent_pkg::*;
import ram_scoreboard_pkg::*;
import ram_coverage_pkg::*;
`include "uvm_macros.svh"

class ram_env extends uvm_env;
    `uvm_component_utils(ram_env)
    ram_agent agt;
    ram_scoreboard sb;
    ram_coverage cov;
    function new(string name="ram_env",uvm_component parent=null);
        super.new(name,parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        agt=ram_agent::type_id::create("agt",this);
        sb=ram_scoreboard::type_id::create("sb",this);
        cov=ram_coverage::type_id::create("cov",this);
    endfunction: build_phase

    function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        agt.agt_ap.connect(sb.sb_export);
        agt.agt_ap.connect(cov.cov_export);
    endfunction

```

```
endclass  
endpackage
```

2.13 Test

```
package ram_test_pkg;  
import ram_env_pkg::*;  
import ram_config_pkg::*;  
import uvm_pkg::*;  
import ram_seq_pkg::*;  
import ram_agent_pkg::*;  
import ram_sequencer_pkg::*;  
`include "uvm_macros.svh"  
  
class ram_test extends uvm_test;  
    `uvm_component_utils(ram_test)  
    ram_env env;  
    ram_config ram_cfg;  
    virtual ram_if ram_vif;  
    ram_main_sequence main_seq;  
    ram_reset_sequence reset_seq;  
    function new(string name="ram_test",uvm_component parent=null);  
        super.new(name,parent);  
    endfunction  
    function void build_phase(uvm_phase phase);  
        super.build_phase(phase);  
        env=ram_env::type_id::create("env",this);  
        ram_cfg=ram_config::type_id::create("ram_cfg");  
        main_seq=ram_main_sequence::type_id::create("main_seq");  
        reset_seq=ram_reset_sequence::type_id::create("reset_seq");  
        if(!uvm_config_db #(virtual ram_if) :: get(this,"", "RAM_IF",ram_cfg.ram_vif))  
begin  
            `uvm_fatal("build_phase","unable to get configuration object");  
        end  
        uvm_config_db #(ram_config)::set(this,"*", "CFG",ram_cfg);  
    endfunction  
  
    task run_phase(uvm_phase phase);  
        super.run_phase(phase);  
        phase.raise_objection(this);  
        `uvm_info("run_phase","reset asserted ",UVM_LOW);  
        reset_seq.start(env.agt.sqr);  
        `uvm_info("run_phase","reset deasserted ",UVM_LOW);  
        `uvm_info("run_phase","stimulus generation",UVM_LOW);
```



```

    main_seq.start(env.agt.sqr);
    `uvm_info("run_phase","stimulus generation ended",UVM_LOW);
    phase.drop_objection(this);
endtask: run_phase
endclass: ram_test
endpackage

```

2.14 Top

```

import uvm_pkg::*;
`include "uvm_macros.svh"
import ram_test_pkg::*;

module top();

    bit clk;

    // Clock generator
    initial begin
        clk = 0;
        forever #1 clk = ~clk;
    end

    ram_if ramif(clk);
    RAM dut (
        .clk      (clk),
        .rst_n     (ramif.rst_n),
        .din       (ramif.din),
        .rx_valid  (ramif.rx_valid),
        .dout      (ramif.dout),
        .tx_valid  (ramif.tx_valid)
    );
    RAM_golden golden(
        .clk      (clk),
        .rst_n     (ramif.rst_n),
        .din       (ramif.din),
        .rx_valid  (ramif.rx_valid),
        .dout      (ramif.dout_ref),
        .tx_valid  (ramif.tx_valid_ref)
    );
    // === Bind SVA module to DUT ===
    bind RAM ram_sva ram_inst(
        .clk      (clk),
        .rst_n     (ramif.rst_n),
        .din       (ramif.din),

```

```

        .rx_valid(ramif.rx_valid),
        .dout      (ramif.dout),
        .tx_valid(ramif.tx_valid)
    );

    initial begin
        uvm_config_db#(virtual ram_if)::set(null,"uvm_test_top","RAM_IF",ramif);
        run_test("ram_test");
    end

endmodule

```

2.15 Adjusted RAM design

```

module RAM (din,clk,rst_n,rx_valid,dout,tx_valid);

input      [9:0] din;
input      clk, rst_n, rx_valid;

output reg [7:0] dout;
output reg      tx_valid;

reg [7:0] MEM [255:0];

reg [7:0] Rd_Addr, Wr_Addr;

always @(posedge clk) begin

    if (~rst_n) begin
        dout <= 0;
        tx_valid <= 0;
        Rd_Addr <= 0;
        Wr_Addr <= 0;
    end
    else begin
        if (rx_valid) begin
            case (din[9:8])
                2'b00 :begin
                    Wr_Addr <= din[7:0];
                    tx_valid <= 0;
                end
                2'b01 :begin
                    MEM[Wr_Addr] <= din[7:0];
                    tx_valid <= 0;
                end
            endcase
        end
    end
end

```

```

        end
        2'b10 : begin
            Rd_Addr <= din[7:0];
            tx_valid <= 0;
        end
        2'b11 : begin
            dout <= MEM[Rd_Addr]; // this was wr addr
            tx_valid <= 1;
        end

        default : dout <= 0;
    endcase
end
end
endmodule

```

2.16 Golden Model

```

module RAM_golden (clk,rst_n,rx_valid,din,dout,tx_valid);
parameter MEM_DEPTH=256;
parameter ADDR_SIZE=8;
input [9:0] din;
input clk,rst_n,rx_valid;
output reg [7:0] dout;
output reg tx_valid;
reg[7:0] mem[MEM_DEPTH-1:0];
reg [ADDR_SIZE-1:0] w_addr, r_addr;
always@(posedge clk ) begin
    if(~rst_n) begin
        dout<=0;
        tx_valid<=0;
        w_addr <= 0;
        r_addr <= 0;
    end
    else begin
        if(rx_valid) begin
            if(~din[9]) begin
                case(din[8])
                    0: begin
                        w_addr<=din[7:0] ;
                        tx_valid<=0;

```

```

        end
        1: begin
            mem[w_addr]<=din[7:0];
            tx_valid<=0;
        end
    endcase
end
else if(din[9]) begin
    case(din[8])
        0: begin
            r_addr<=din[7:0] ;
            tx_valid<=0;
        end
        1: begin
            dout<=mem[r_addr];
            tx_valid<=1;
        end
    endcase
end
end
end
endmodule

```

2.17 Verification Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
RAM_REQ_1	When reset (rst_n) is low, outputs dout and tx_valid should be driven to 0.	Directed: rst_n=0	Cover property reset_clear	Checked by assertion property reset_clear
RAM_REQ_2	During WA, WD, or RA operations (din[9:8] = 00, 01, or 10), tx_valid must remain 0.	Constrained random (main sequence) — ensures all 3 opcodes appear	Cover property tx_valid_off; coverpoints din_cp and tx_valid_cp in covcode	Checked by assertion property tx_valid_off
RAM_REQ_3	During RD operation (din[9:8] = 11), tx_valid should pulse high then return low.	Constrained random or directed: din=10'b111xxxx, rx_valid=1	Cover property tx_valid_on; coverpoint cross_rddata_txvalid.valid_read	Checked by assertion property tx_valid_on
RAM_REQ_4	Every WA command (din[9:8]=00) must eventually be followed by a WD command (din[9:8]=01).	Covered in ram_write_read_sequence transitions	Cover property write_operation; coverpoint trans_cp.wraddr_to_wrdata	Checked by assertion property write_operation
RAM_REQ_5	Every RA command (din[9:8]=10) must eventually be followed by an RD command (din[9:8]=11).	Covered in ram_write_read_sequence transitions	Cover property read_operation; coverpoint trans_cp.rdaddr_to_rddata	Checked by assertion property read_operation
RAM_REQ_6	Verify all valid transitions between operations: WA→WD→RA→RD sequence.	Randomized sequence ram_write_read_sequence with 1000 operations	Coverpoint trans_cp.full_sequence	Verified through coverage group sampling in ram_coverage.sv
RAM_REQ_7	System resilience: Design should operate correctly after a mid-simulation reset.	Directed test: deassert rst_n mid-run	Cover property reset_clear (re-activation after reset)	Checked by assertion property reset_clear
RAM_REQ_8	Ensure correct behavior of data/address fields during write/read.	Randomized din[7:0] under rx_valid=1	Coverpoints wr_data_cp and rd_data_cp	Validated through functional coverage bins and scoreboard comparison

2.18 Bug Report

Bug Number 1: Wrong Memory Address Used During Read Operation

a) Design Input for Bug to Appear:

1. Reset the module ($\text{rst_n} = 0 \rightarrow 1$).
2. Send a write address command ($\text{din}[9:8] = 2'b00$) with address A.
3. Send a write data command ($\text{din}[9:8] = 2'b01$) with data D.
4. Send a read address command ($\text{din}[9:8] = 2'b10$) with a different address B.
5. Send a read command ($\text{din}[9:8] = 2'b11$).

b) Expected Behavior:

dout should output data stored at MEM[B], corresponding to the read address.

c) Observed Behavior:

dout instead outputs data from MEM[A], the last written address.

Bug Number 2: tx_valid Signal Behavior Not Controlled Inside Operation Cases

a) Design Input for Bug to Appear:

Perform any of the following operations:

- Write Address ($\text{din}[9:8] = 2'b00$)
- Write Data ($\text{din}[9:8] = 2'b01$)
- Read Address ($\text{din}[9:8] = 2'b10$)
- Read Data ($\text{din}[9:8] = 2'b11$)

b) Expected Behavior:

tx_valid should:

- Stay **low** during WA, WD, and RA operations.
- Assert **high** only during RD operation to signal valid output data.

c) Observed Behavior:

In the buggy design, tx_valid is updated outside the case statement based only on din[9:8] bits.

This causes tx_valid to briefly assert or deassert at the wrong times, potentially misaligning with actual data output timing.

2.19 Assertion Table

Feature	Assertion
Reset clears outputs	`@(posedge clk) (!rst_n)
During Write Address (WA), Write Data (WD), or Read Address (RA), tx_valid remains low	`@(posedge clk) disable iff (!rst_n) (rx_valid && (din[9:8] inside {2'b00, 2'b01, 2'b10}))
During Read Data (RD) operation, tx_valid must assert then deassert	`@(posedge clk) disable iff (!rst_n) (rx_valid && din[9:8] == 2'b11)
A Write Address (WA) command must eventually be followed by a Write Data (WD) command	`@(posedge clk) disable iff (!rst_n) (rx_valid && din[9:8] == 2'b00)
A Read Address (RA) command must eventually be followed by a Read Data (RD) command	`@(posedge clk) disable iff (!rst_n) (rx_valid && din[9:8] == 2'b10)

2.20 Coverage Report

Assertion Coverage:						
Assertions	5	5	0	100.00%		

Name	File(Line)	Failure Count	Pass Count			

/top/dut/ram_inst/assert_read_operation	ram_sva.sv(45)	0	1			
/top/dut/ram_inst/assert_write_operation	ram_sva.sv(44)	0	1			
/top/dut/ram_inst/assert_tx_valid_on	ram_sva.sv(43)	0	1			
/top/dut/ram_inst/assert_tx_valid_off	ram_sva.sv(42)	0	1			
/top/dut/ram_inst/assert_reset_clear	ram_sva.sv(41)	0	1			
Directive Coverage:						
Directives	5	5	0	100.00%		
DIRECTIVE COVERAGE:						

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status

/top/dut/ram_inst/cover_read_operation	ram_sva	Verilog	SVA	ram_sva.sv(52)	809	Covered
/top/dut/ram_inst/cover_write_operation	ram_sva	Verilog	SVA	ram_sva.sv(51)	1259	Covered
/top/dut/ram_inst/cover_tx_valid_on	ram_sva	Verilog	SVA	ram_sva.sv(50)	4166	Covered
/top/dut/ram_inst/cover_tx_valid_off	ram_sva	Verilog	SVA	ram_sva.sv(49)	4794	Covered
/top/dut/ram_inst/cover_reset_clear	ram_sva	Verilog	SVA	ram_sva.sv(48)	1516	Covered
=====						
=== Instance: /top/dut						
=== Design Unit: work.RAM						
=====						
Branch Coverage:						
Enabled Coverage	Bins	Hits	Misses	Coverage		

Branches	8	8	0	100.00%		
=====Branch Details=====						
Branch Coverage for instance /top/dut						
Line	Item	Count	Source			

File RAM.v						
-----IF Branch-----						
14		14997	Count coming in to IF			
14	1	1516	if (~rst_n) begin			
20	1	13481	else begin			
Branch totals: 2 hits of 2 branches = 100.00%						
-----IF Branch-----						
21		13481	Count coming in to IF			

Branch Coverage: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
21                               13481   Count coming in to IF
21             1                10765   if (cx_valid) begin
                               2716   All False Count
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----CASE Branch-----
22                               10765   Count coming in to CASE
23             1                2167   2'b00 : wr_addr <= din[7:0];
24             1                2122   2'b01 : MEM[wr_addr] <= din[7:0];
25             1                1048   2'b10 : rd_addr <= din[7:0];
26             1                5428   2'b11 : dout <= MEM[rd_addr]; // this was wr_addr

Branch totals: 4 hits of 4 branches = 100.00%
```

```
Expression Coverage:
Enabled Coverage      Bins   Covered   Misses   Coverage
-----
Expressions           3       3         0   100.00%
```

=====Expression Details=====

Expression Coverage for instance /top/dut --

```
File RAM.v
-----Focused Expression View-----
Line   30 Item 1 ((din[9] && din[8]) && cx_valid)
Expression totals: 3 of 3 input terms covered = 100.00%
```

Input Term	Covered	Reason for no coverage	Hint
din[9]	Y		
din[8]	Y		
cx_valid	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	din[9]_0	-
Row 2:	1	din[9]_1	(cx_valid && din[8])
Row 3:	1	din[8]_0	din[9]
Row 4:	1	din[8]_1	(cx_valid && din[9])
Row 5:	1	rx_valid_0	(din[9] && din[8])
Row 6:	1	rx_valid_1	(din[9] && din[8])

```
Statement Coverage:
Enabled Coverage      Bins   Hits   Misses   Coverage
-----
Statements           10      10         0   100.00%
```

=====Statement Details=====

Statement Coverage for instance /top/dut --

Line	Item	Count	Source
File RAM.v			
1			module RAM (din,clk,cst_n,cx_valid,dout,tx_valid);
2			
3			input [9:0] din;
4			input clk, cst_n, cx_valid;
5			
6			output reg [7:0] dout;
7			output reg tx_valid;
8			
9			reg [7:0] MEM [255:0];
10			

Ln 90, Col 54 | 143,472 characters | Plain text

Covergroup Coverage:

Covergroups	1	na	na	100.00%
Coverpoints/Crosses	8	na	na	na
Covergroup Bins	28	28	0	100.00%

Covergroup	Metric	Goal	Bins	Status
TYPE /ram_coverage_pkg/ram_coverage/covcode	100.00%	100	-	Covered
covered/total bins:	28	28	-	
missing/total bins:	0	28	-	
% Hit:	100.00%	100	-	
Coverpoint din_cp	100.00%	100	-	Covered
covered/total bins:	4	4	-	
missing/total bins:	0	4	-	
% Hit:	100.00%	100	-	
Coverpoint wr_data_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Coverpoint rd_data_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Coverpoint trans_cp	100.00%	100	-	Covered
covered/total bins:	4	4	-	
missing/total bins:	0	4	-	
% Hit:	100.00%	100	-	
Coverpoint rx_valid_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
Coverpoint tx_valid_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
Cross cross_din_cxvalid	100.00%	100	-	Covered
covered/total bins:	5	5	-	
missing/total bins:	0	5	-	
% Hit:	100.00%	100	-	
Cross cross_rddata_txvalid	100.00%	100	-	Covered
covered/total bins:	5	5	-	
missing/total bins:	0	5	-	
% Hit:	100.00%	100	-	
Covergroup instance \ram_coverage_pkg::ram_coverage::covcode	100.00%	100	-	Covered
covered/total bins:	28	28	-	
missing/total bins:	0	28	-	
% Hit:	100.00%	100	-	
Coverpoint din_cp	100.00%	100	-	Covered
covered/total bins:	4	4	-	
missing/total bins:	0	4	-	
% Hit:	100.00%	100	-	
bin wc_addr	3099	1	-	Covered
bin wc_data	3124	1	-	Covered
bin rd_addr	1924	1	-	Covered
bin rd_data	6854	1	-	Covered
Coverpoint wr_data_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin wr_data_0	20	1	-	Covered
bin wc_data_max	13	1	-	Covered
bin wc_data_min	12	1	-	Covered
default bin wr_data_default	3079	-	-	Occurred
Coverpoint rd_data_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	

Coverpoint rd_data_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin rd_data_0	26	1	-	Covered
bin rd_data_max	24	1	-	Covered
bin rd_data_min	28	1	-	Covered
default bin rd_data_default	6776		-	Occurred
Coverpoint txns_cp	100.00%	100	-	Covered
covered/total bins:	4	4	-	
missing/total bins:	0	4	-	
% Hit:	100.00%	100	-	
bin wraddr_to_wrddata	610	1	-	Covered
bin wrdata_to_rdaddr	417	1	-	Covered
bin rdaddr_to_rddata	878	1	-	Covered
bin full_sequence	36	1	-	Covered
Coverpoint rx_valid_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin low	3007	1	-	Covered
bin high	11994	1	-	Covered
Coverpoint tx_valid_cp	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin low	9571	1	-	Covered
bin high	5430	1	-	Covered
Cross cross_din_rxvalid	100.00%	100	-	Covered
covered/total bins:	5	5	-	
missing/total bins:	0	5	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin active_cross	11994	1	-	Covered
bin <rd_data.low>	801	1	-	Covered
bin <wr_data.low>	757	1	-	Covered
bin <rd_addr.low>	762	1	-	Covered
bin <wr_addr.low>	687	1	-	Covered
Cross cross_rddata_txvalid	100.00%	100	-	Covered
covered/total bins:	5	5	-	
missing/total bins:	0	5	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin valid_read	5430	1	-	Covered
bin <rd_data.low>	1424	1	-	Covered
bin <wr_data.low>	3124	1	-	Covered
bin <rd_addr.low>	1924	1	-	Covered
bin <wr_addr.low>	3099	1	-	Covered
Illegal and Ignore Bins:				
illegal_bin wr_high	0		-	ZERO
illegal_bin wr_high	0		-	ZERO

2.21 Questasim

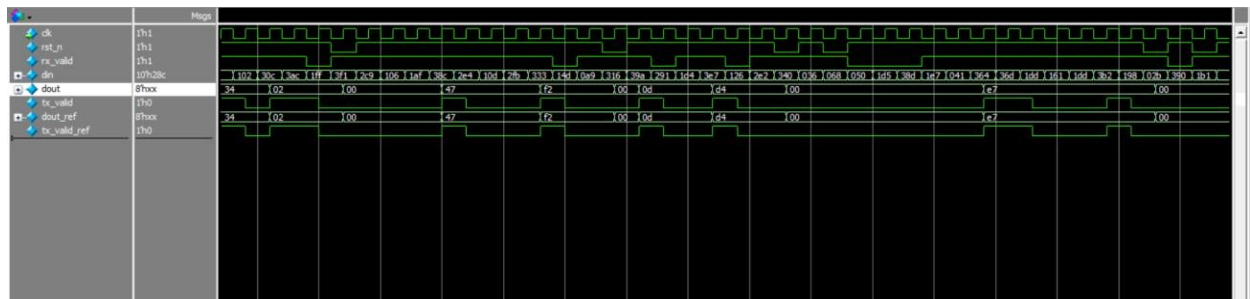
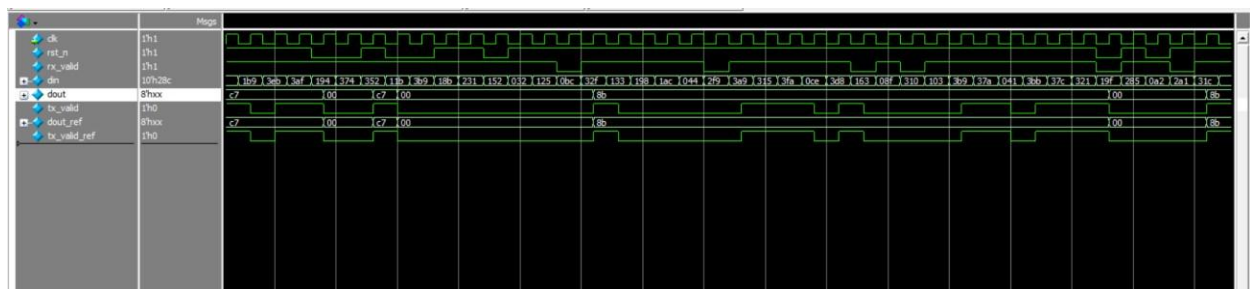
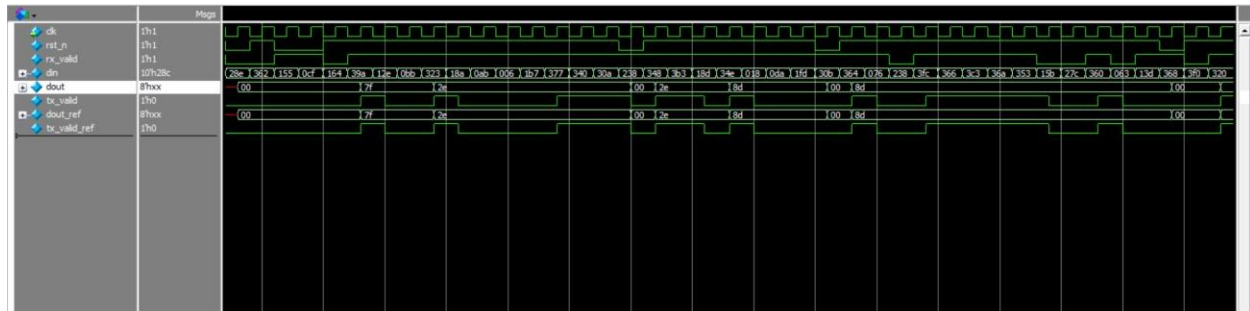
Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage
/ram_coverage_pk...		100.00%						
TYPE covcode		100.00%	100	100.00...		✓	auto(0)	

/top/dut/ram_inst/...	Concurrent	SVA	on	0	1	-	0B	0B
/top/dut/ram_inst/...	Concurrent	SVA	on	0	1	-	0B	0B
/top/dut/ram_inst/...	Concurrent	SVA	on	0	1	-	0B	0B
/top/dut/ram_inst/...	Concurrent	SVA	on	0	1	-	0B	0B
/top/dut/ram_inst/...	Concurrent	SVA	on	0	1	-	0B	0B

```

# UVM_INFO ram_test.sv(38) @ 2: uvm_test_top [run_phase] reset deasserted
# UVM_INFO ram_test.sv(39) @ 2: uvm_test_top [run_phase] stimulus generation
# UVM_INFO ram_test.sv(41) @ 30002: uvm_test_top [run_phase] stimulus generation ended
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 30002: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO ram_scoreboard.sv(49) @ 30002: uvm_test_top.env.sb [report_phase] total successful transactions:15001
# UVM_INFO ram_scoreboard.sv(50) @ 30002: uvm_test_top.env.sb [report_phase] total failed transactions:0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 10
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [ Questa UVM ] 2
# [ RNTSTI ] 1
# [ TEST_DONE ] 1
# [ report_phase ] 2
# [ run_phase ] 4

```



2.22 DO File

```
vlib work
vlog -sv -f src_list.list -cover bcesf
vsim -coverage -voptargs=+acc work.top -classdebug -uvmcontrol=all
coverage save top.ucdb -onexit
run -all|
coverage exclude -src RAM.v -line 41 -code s
coverage exclude -src RAM.v -line 41 -code b
```

3.0 WRAPPER

3.1 Wrapper

```
module wrapper (MOSI, MISO, SS_n, clk, rst_n);

// Input and output ports
input  MOSI, clk, rst_n, SS_n;
output MISO;

wire [9:0] rx_data;
wire rx_valid;
wire [7:0] tx_data;
wire tx_valid;
// Instantiate SLAVE module
SLAVE slave (
    .MOSI(MOSI),
    .MISO(MISO),
    .SS_n(SS_n),
    .clk(clk),
    .rst_n(rst_n),
    .rx_data(rx_data),
    .rx_valid(rx_valid),
    .tx_data(tx_data),
    .tx_valid(tx_valid)
);

// Instantiate RAM module
RAM memory (
    .clk(clk),
    .rst_n(rst_n),
    .din(rx_data),
    .rx_valid(rx_valid),
    .dout(tx_data),
    .tx_valid(tx_valid)
);
```

```

// Assertions
`ifdef WRAPPER_SIM
// Check reset values
always_comb begin
    if (!rst_n) begin
        wrap_rst_tx_valid_a: assert final (tx_valid == 0);
        wrap_rst_rx_valid_a: assert final (rx_valid == 0);
        wrap_rst_tx_data_a: assert final (tx_data == 0);
        wrap_rst_rx_data_a: assert final (rx_data == 0);
        wrap_rst_miso_a: assert final (MISO == 0);
    end
end

// Check MISO stability during non-read operations
property wrap_miso_stable_pr;
    @(posedge clk) disable iff(!rst_n) (SS_n == 0) ##1 (!SS_n &&
        ({past(MOSI, 10), past(MOSI, 9), past(MOSI, 8)} != 3'b111)) |> stable(MISO);
endproperty
wrap_miso_stable_a: assert property (wrap_miso_stable_pr);
wrap_miso_stable_cov: cover property (wrap_miso_stable_pr);
// Check rx_valid
property wrap_rx_valid_pr;
    @(posedge clk) disable iff(!rst_n) (SS_n) ##1 (!SS_n) [*12:22] |> (rx_valid);
endproperty
wrap_rx_valid_a: assert property (wrap_rx_valid_pr);
wrap_rx_valid_cov: cover property (wrap_rx_valid_pr);

// Check tx_valid
property wrap_tx_valid_pr;
    @(posedge clk) disable iff(!rst_n) (SS_n) ##1 (!SS_n) [*12:22] ##1 (!SS_n) &&
        ({past(MOSI, 10), past(MOSI, 9), past(MOSI, 8)} == 3'b111) |> (tx_valid);
endproperty
wrap_tx_valid_a: assert property (wrap_tx_valid_pr);
wrap_tx_valid_cov: cover property (wrap_tx_valid_pr);

// Check rx_data
property wrap_rx_data_pr;
    @(posedge clk) disable iff(!rst_n) (SS_n) ##1 (!SS_n) [*12:22] |> (rx_valid |> (rx_data ==
        {past(MOSI, ADDR_SIZE+1), past(MOSI, ADDR_SIZE), past(MOSI, ADDR_SIZE-1), past(MOSI, ADDR_SIZE-2),
        past(MOSI, ADDR_SIZE-3), past(MOSI, ADDR_SIZE-4), past(MOSI, ADDR_SIZE-5), past(MOSI, ADDR_SIZE-6),
        past(MOSI, ADDR_SIZE-7), past(MOSI, ADDR_SIZE-8)}));
endproperty
wrap_rx_data_a: assert property (wrap_rx_data_pr);
wrap_rx_data_cov: cover property (wrap_rx_data_pr);
`endif
endmodule

```

Activate
Go to Setting

3.2 Wrapper_Top

```

1  import uvm_pkg::*;
2  `include "uvm_macros.svh"
3  import wrapper_test_pkg::*;
4
5  module wrapper_top ();
6      bit clk;
7      localparam MEM_DEPTH = 256;
8
9      // Clock generation
10     initial begin
11         clk = 0;
12         forever #1 clk = ~clk;
13     end
14
15     // Instantiate interfaces
16     wrapper_if wrapperif (clk);
17     ram_if ramif (clk);
18     slave_if slaveif (clk);
19
20     wrapper_DUT (
21         .MOSI(wrapperif.MOSI),
22         .MISO(wrapperif.MISO),
23         .SS_n(wrapperif.SS_n),
24         .clk(wrapperif.clk),
25         .rst_n(wrapperif.rst_n)
26     );
27
28     RAM_golden #(
29         .MEM_DEPTH(MEM_DEPTH)
30     ) ram_ref (
31         .clk(wrapperif.clk),
32         .rst_n(wrapperif.rst_n),
33         .rx_valid(slaveif.rx_valid_ref),
34         .din(slaveif.rx_data_ref),
35         .dout(ramif.dout_ref),
36         .tx_valid(ramif.tx_valid_ref)
37     );
38

```

```

39     slave_golden_slave_ref (
40         .MOSI(wrapperif.MOSI),
41         .MISO(slaveif.MISO_ref),
42         .SS_n(wrapperif.SS_n),
43         .clk(wrapperif.clk),
44         .rst_n(wrapperif.rst_n),
45         .rx_data(slaveif.rx_data_ref),
46         .rx_valid(slaveif.rx_valid_ref),
47         .tx_data(ramif.dout_ref),
48         .tx_valid(ramif.tx_valid_ref)
49     );
50
51
52     // Initial block for configuration and test run
53     initial begin
54         $readmemh("RAM_data.dat", DUT.memory.MEM, 0, 255); // Fixed memory load path
55         uvm_config_db #(virtual wrapper_if)::set(null, "uvm_test_top", "wrapper_if", wrapperif);
56         uvm_config_db #(virtual slave_if)::set(null, "uvm_test_top", "slave_vif", slaveif);
57         uvm_config_db #(virtual ram_if)::set(null, "uvm_test_top", "ram_vif", ramif);
58         run_test("wrapper_test");
59     end
60
61     assign slaveif.rst_n    = DUT.rst_n;
62     assign slaveif.SS_n    = DUT.SS_n;
63     assign slaveif.MOSI    = DUT.MOSI;
64     assign slaveif.rx_valid = DUT.slave.rx_valid;
65     assign slaveif.rx_data = DUT.slave.rx_data;
66     assign slaveif.tx_valid = DUT.memory.tx_valid;
67     assign slaveif.tx_data = DUT.memory.dout;
68     assign slaveif.MISO    = DUT.MISO;
69
70     assign ramif.rst_n    = DUT.rst_n;
71     assign ramif.rx_valid = DUT.slave.rx_valid;
72     assign ramif.din      = DUT.slave.rx_data;
73     assign ramif.tx_valid = DUT.memory.tx_valid;
74     assign ramif.dout     = DUT.memory.dout;
75
76 endmodule

```

3.3 Wrapper_Interface

```

1
2 interface wrapper_if (input clk);
3     parameter MEM_DEPTH = 256;
4     parameter ADDR_SIZE=8;
5     logic rst_n, SS_n, MOSI, MISO;
6 endinterface

```

3.4 Wrapper_Sequence_Item

```
1 package wrapper_seq_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import shared_pkg::*;
5
6 typedef enum bit [1 : 0] {WA = 2'b00, WD = 2'b01, RA = 2'b10, RD = 2'b11} cmd_e;
7
8 class wrapper_seq_item extends uvm_sequence_item;
9     `uvm_object_utils(wrapper_seq_item)
10
11     rand bit rst_n;
12     bit SS_n;
13     bit MOSI;
14     rand bit [9:0] mosi_bits;
15     bit tx_valid;
16     bit MISO;
17     cmd_e cmd;
18
19     constraint rst_c { rst_n dist { 1 := 90, 0 := 10 }; }
20
21     constraint mosi_bits_c {
22         mosi_bits[9:8] inside {WA, WD, RA, RD};
23         mosi_bits[7:0] dist { [0:255] := 1 };
24     }
25
26     function void post_randomize();
27         cmd = cmd_e'(mosi_bits[9:8]);
28         tx_valid = (cmd == RD);
29         if (!rst_n) cs_cov = IDLE_cov;
30         else case (mosi_bits[9:8])
31             WA: cs_cov = WA_cov;
32             WD: cs_cov = WD_cov;
33             RA: cs_cov = RA_cov;
34             RD: cs_cov = RD_cov;
35         endcase
36     endfunction
37
38     function new(string name = "wrapper_seq_item");
39         super.new(name);
40         SS_n = 1'b1;
41         MOSI = 1'b0;
42         tx_valid = 1'b0;
43         cmd = WA;
44     endfunction
45
```

```
46     function string convert2string();
47         return $sformatf("%s rst_n=%0b, SS_n=%0b, MOSI=%0b, mosi_bits=%b, tx_valid=%0b, MISO=%0b, cmd=%s",
48             super.convert2string(), rst_n, SS_n, MOSI, mosi_bits, tx_valid, MISO, cmd.name());
49     endfunction
50
51     function string convert2string_stimulus();
52         return $sformatf("rst_n=%0b, SS_n=%0b, MOSI=%0b, mosi_bits=%b, tx_valid=%0b, cmd=%s",
53             rst_n, SS_n, MOSI, mosi_bits, tx_valid, cmd.name());
54     endfunction
55 endclass
56 endpackage
57
```

3.5 Shared_Package

```
1 package shared_pkg;
2     localparam MEM_DEPTH = 256;
3     localparam ADDR_SIZE = $clog2(MEM_DEPTH);
4
5     // State machine states for design
6     typedef enum bit [2 : 0] {IDLE, CHK_CMD, WRITE, READ_ADD, READ_DATA} STATE_e;
7
8     // Coverage-specific command states
9     typedef enum bit [2 : 0] {IDLE_cov = 3'b000, WA_cov = 3'b001, WD_cov = 3'b010, RA_cov = 3'b110, RD_cov = 3'b111} CMD_cov_e;
10     CMD_cov_e cs_cov;
11
12 endpackage
13
```


3.6 Wrapper_Sequence

```
1 package wrapper_sequence_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import wrapper_seq_item_pkg::*;
5
6 class wrapper_reset_sequence extends uvm_sequence #(wrapper_seq_item);
7     `uvm_object_utils(wrapper_reset_sequence)
8     wrapper_seq_item seq_item;
9
10    function new(string name = "wrapper_reset_sequence");
11        super.new(name);
12    endfunction
13
14    task body;
15        seq_item = wrapper_seq_item::type_id::create("seq_item");
16        start_item(seq_item);
17        seq_item.rst_n = 0;
18        seq_item.ss_n = 1;
19        seq_item.MOSI = 0;
20        seq_item.mosi_bits = 0;
21        finish_item(seq_item);
22    endtask
23 endclass
24
25 class wrapper_write_only_sequence extends uvm_sequence #(wrapper_seq_item);
26     `uvm_object_utils(wrapper_write_only_sequence)
27     wrapper_seq_item seq_item;
28     bit [1:0] last_cmd = WA;
29
30    function new(string name = "wrapper_write_only_sequence");
31        super.new(name);
32    endfunction
33
34    task body;
35        repeat(50) begin
36            seq_item = wrapper_seq_item::type_id::create("seq_item");
37            start_item(seq_item);
38            assert(seq_item.randomize() with {
39                mosi_bits[9:8] inside {WA, WD};
40                if (last_cmd inside {WA, WD}) mosi_bits[9:8] inside {WA, WD};
41                else mosi_bits[9:8] == WA;
42            });
43            finish_item(seq_item);
44            last_cmd = seq_item.mosi_bits[9:8];
45        end
46    endtask
47 endclass
48
```



```

class wrapper_read_only_sequence extends uvm_sequence #(wrapper_seq_item);
  `uvm_object_utils(wrapper_read_only_sequence)
  wrapper_seq_item seq_item;
  bit [1:0] last_cmd = RA;

  function new(string name = "wrapper_read_only_sequence");
    super.new(name);
  endfunction

  task body;
    repeat(50) begin
      seq_item = wrapper_seq_item::type_id::create("seq_item");
      start_item(seq_item);
      assert(seq_item.randomize() with {
        mosi_bits[9:8] inside {RA, RD};
        if (last_cmd inside {RA, RD}) mosi_bits[9:8] inside {RA, RD};
        else mosi_bits[9:8] == RA;
      });
      finish_item(seq_item);
      last_cmd = seq_item.mosi_bits[9:8];
    end
  endtask
endclass

class wrapper_write_read_sequence extends uvm_sequence #(wrapper_seq_item);
  `uvm_object_utils(wrapper_write_read_sequence)
  wrapper_seq_item seq_item;
  bit [1:0] last_cmd = WA;

  function new(string name = "wrapper_write_read_sequence");
    super.new(name);
  endfunction

  task body;
    repeat(50) begin
      seq_item = wrapper_seq_item::type_id::create("seq_item");
      start_item(seq_item);
      assert(seq_item.randomize() with {
        mosi_bits[9:8] inside {WA, WD, RA, RD};
        if (last_cmd inside {WA, WD}) mosi_bits[9:8] inside {RA, RD};
        else if (last_cmd inside {RA, RD}) mosi_bits[9:8] inside {WA, WD};
        else mosi_bits[9:8] == WA;
      });
      finish_item(seq_item);
      last_cmd = seq_item.mosi_bits[9:8];
    end
  endtask
endclass

```

3.7 Wrapper_Sequencer

```

1 package wrapper_sequencer_pkg;
2   import uvm_pkg::*;
3   `include "uvm_macros.svh"
4   import wrapper_seq_item_pkg::*;
5
6   class wrapper_sequencer extends uvm_sequencer #(wrapper_seq_item);
7     `uvm_component_utils(wrapper_sequencer)
8
9     function new(string name = "wrapper_sequencer", uvm_component parent = null);
10       super.new(name, parent);
11     endfunction
12   endclass
13 endpackage

```

3.8 Wrapper_Configuration_Object

```
1 package wrapper_config_obj_pkg;
2     import uvm_pkg::*;
3     import shared_pkg::*;
4     `include "uvm_macros.svh"
5
6     class wrapper_config_obj extends uvm_object;
7         `uvm_object_utils(wrapper_config_obj)
8
9         virtual wrapper_if wrapper_config_if;
10        uvm_active_passive_enum active;
11
12        function new(string name = "wrapper_config_obj");
13            super.new(name);
14        endfunction
15    endclass
16
17 endpackage
```

3.9 Wrapper_Driver

```
package wrapper_driver_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import shared_pkg::*;
    import wrapper_seq_item_pkg::*;
    import wrapper_config_obj_pkg::*;

    class wrapper_driver extends uvm_driver #(wrapper_seq_item);
        `uvm_component_utils(wrapper_driver)

        virtual wrapper_if #(MEM_DEPTH) wrapper_driver_if;
        wrapper_seq_item stim_seq_item;

        function new(string name = "wrapper_driver", uvm_component parent = null);
            super.new(name, parent);
        endfunction

        task automatic drive_reset(bit rst_n_val);
            wrapper_driver_if.rst_n = rst_n_val;
            wrapper_driver_if.SS_n = 1'b1;
            wrapper_driver_if.MOSI = 1'b0;
            @(posedge wrapper_driver_if.clk);
        endtask

        task automatic drive_frame(wrapper_seq_item item);
            // Hold SS_n low while streaming out the 10-bit frame MSB-first
            wrapper_driver_if.SS_n = 1'b0;
            for (int bit_idx = 9; bit_idx >= 0; bit_idx--) begin
                @(negedge wrapper_driver_if.clk);
                wrapper_driver_if.MOSI = item.mosi_bits[bit_idx];
                @(posedge wrapper_driver_if.clk);
            end

            // Allow the slave one extra beat to assert rx_valid while SS_n remains low
            @(negedge wrapper_driver_if.clk);
            wrapper_driver_if.MOSI = 1'b0;
            @(posedge wrapper_driver_if.clk);

            // For read commands keep clocking the interface so the slave can drive MISO
            if (item.cmd == RD) begin
                for (int rd_bit = 7; rd_bit >= 0; rd_bit--) begin
                    @(negedge wrapper_driver_if.clk);
                    wrapper_driver_if.MOSI = 1'b0;
                    @(posedge wrapper_driver_if.clk);
                end
            end
        end
    end
```

3.10 Wrapper_Monitor

```
1 package wrapper_monitor_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import shared_pkg::*;
5 import wrapper_seq_item_pkg::*;
6
7 class wrapper_monitor extends uvm_monitor;
8     `uvm_component_utils(wrapper_monitor)
9
10     virtual wrapper_if #(MEM_DEPTH) wrapper_monitor_if;
11     wrapper_seq_item rsp_seq_item;
12     uvm_analysis_port #(wrapper_seq_item) mon_ap;
13
14     function new(string name = "wrapper_monitor", uvm_component parent = null);
15         super.new(name, parent);
16     endfunction
17
18     function void build_phase(uvm_phase phase);
19         super.build_phase(phase);
20         mon_ap = new("mon_ap", this);
21     endfunction
22
23     // Track current command bits for diagnostics/coverage
24     bit [9:0] sampled_bits;
25     int bit_count;
26     bit prev_ss_n;
27
28     task run_phase(uvm_phase phase);
29         super.run_phase(phase);
30         sampled_bits = '0;
31         bit_count = 0;
32         prev_ss_n = 1'b1;
33
34         forever begin
35             rsp_seq_item = wrapper_seq_item::type_id::create("rsp_seq_item");
36             @(posedge wrapper_monitor_if.clk);
37
38             // Reset per-frame book-keeping on reset deassertion or SS_n rising edge
39             if (!wrapper_monitor_if.rst_n) begin
40                 sampled_bits = '0;
41                 bit_count = 0;
42             end
43
44             if (prev_ss_n && !wrapper_monitor_if.SS_n) begin
45                 sampled_bits = '0;
46                 bit_count = 0;
47             end
```

```
4         forever begin
5             rsp_seq_item = wrapper_seq_item::type_id::create("rsp_seq_item");
6             @(posedge wrapper_monitor_if.clk);
7
8             // Reset per-frame book-keeping on reset deassertion or SS_n rising edge
9             if (!wrapper_monitor_if.rst_n) begin
10                 sampled_bits = '0;
11                 bit_count = 0;
12             end
13
14             if (prev_ss_n && !wrapper_monitor_if.SS_n) begin
15                 sampled_bits = '0;
16                 bit_count = 0;
17             end
18
19             if (!wrapper_monitor_if.SS_n && bit_count < 10) begin
20                 sampled_bits[9 - bit_count] = wrapper_monitor_if.MOSI;
21                 bit_count++;
22             end
23
24             rsp_seq_item.rst_n = wrapper_monitor_if.rst_n;
25             rsp_seq_item.SS_n = wrapper_monitor_if.SS_n;
26             rsp_seq_item.MOSI = wrapper_monitor_if.MOSI;
27             rsp_seq_item.MISO = wrapper_monitor_if.MISO;
28             rsp_seq_item.mosi_bits = sampled_bits;
29             if (bit_count >= 2) begin
30                 rsp_seq_item.cmd = cmd_e'(sampled_bits[9:8]);
31             end
32
33             mon_ap.write(rsp_seq_item);
34             `uvm_info("run_phase", rsp_seq_item.convert2string(), UVM_HIGH)
35
36             prev_ss_n = wrapper_monitor_if.SS_n;
37         end
38     endtask
39 endclass
40 endpackage
```

3.11 Wrapper_Scoreboard

```
1 package wrapper_scoreboard_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import wrapper_seq_item_pkg::*;
5 import shared_pkg::*;
6
7 class wrapper_scoreboard extends uvm_scoreboard;
8     `uvm_component_utils(wrapper_scoreboard)
9
10     uvm_analysis_export #(wrapper_seq_item) sb_export;
11     uvm_tlm_analysis_fifo #(wrapper_seq_item) sb_fifo;
12     wrapper_seq_item seq_item_sb;
13
14     STATE e cs_ref;
15     Logic MISO_ref;
16     bit [ADDR_SIZE + 1 : 0] rx_data_ref;
17     bit [$clog2(ADDR_SIZE) : 0] count_rx_ref, count_tx_ref;
18     bit rd_flag_ref, rx_valid_in_RD, rx_valid_ref, tx_valid_ref;
19     Logic [ADDR_SIZE - 1 : 0] mem_ref [MEM_DEPTH - 1 : 0];
20     bit [ADDR_SIZE - 1 : 0] wa_ref, ra_ref, tx_data_ref;
21
22     int error_count = 0;
23     int correct_count = 0;
24
25     function new(string name = "wrapper_scoreboard", uvm_component parent = null);
26         super.new(name, parent);
27     endfunction
28
29     function void build_phase(uvm_phase phase);
30         super.build_phase(phase);
31         sb_export = new("sb_export", this);
32         sb_fifo = new("sb_fifo", this);
33     endfunction
34
35     function void connect_phase(uvm_phase phase);
36         super.connect_phase(phase);
37         sb_export.connect(sb_fifo.analysis_export);
38     endfunction
39
40     task run_phase(uvm_phase phase);
41         super.run_phase(phase);
42         $readmemh("RAM_data.dat", mem_ref, 0, 255);
43         forever begin
44             sb_fifo.get(seq_item_sb);
45             ref_model(seq_item_sb);
46
47             // Only check MISO when the reference model expects valid read data
48             if (tx_valid_ref) begin
49                 if (seq_item_sb.MISO != MISO_ref) begin
50                     `uvm_error("run_phase", $sformatf("comparsion failed, Transaction received by the DUT:%s While the reference: = %0b",
51                         seq_item_sb.convert2string(), MISO_ref))
52                     error_count++;
53                 end
54             else begin
55                 `uvm_info("run_phase", $sformatf("Correct wrapper tx_data: %s", seq_item_sb.convert2string()), UVM_HIGH)
56                 correct_count++;
57             end
58         end
59     endtask //
60
61     task ref_model(wrapper_seq_item seq_item_chk);
62         // rx_data && MISO
63         if (!seq_item_chk.rst_n) begin
64             cs_ref = IDLE;
65             count_rx_ref = ADDR_SIZE + 2;
66             count_tx_ref = ADDR_SIZE;
67             rd_flag_ref = 0;
68             rx_data_ref = 0;
69             rx_valid_in_RD = 0;
70             MISO_ref = 0;
71             tx_valid_ref = 0;
72             rx_valid_ref = 0;
73         end
74     else begin
75         case (cs_ref)
76             IDLE: begin
77                 count_rx_ref = ADDR_SIZE + 2;
78                 count_tx_ref = ADDR_SIZE;
79                 rx_data_ref = 0;
80                 MISO_ref = 0;
81                 tx_valid_ref = 0;
82                 if (!seq_item_chk.SS_n)
83                     cs_ref = CHK_CMD;
84             end
85             CHK_CMD: begin
86                 if (seq_item_chk.SS_n)
87                     cs_ref = IDLE;
88                 else if (!seq_item_chk.MOSI)
89                     cs_ref = WRITE;
90                 else if (!rd_flag_ref)
91                     cs_ref = READ_ADD;
92         endcase
93     end
94 endtask
```



```

96         WRITE: begin
97             if (seq_item_chk.SS_n)
98                 cs_ref = IDLE;
99                 if (count_rx_ref != 0) begin
100                     rx_data_ref [count_rx_ref - 1] = seq_item_chk.MOSI;
101                     count_rx_ref--;
102                 end
103             end
104         READ_ADD: begin
105             if (seq_item_chk.SS_n) begin
106                 cs_ref = IDLE;
107                 if (count_rx_ref == 0)
108                     rd_flag_ref = 1;
109             end
110             if (count_rx_ref != 0) begin
111                 rx_data_ref [count_rx_ref - 1] = seq_item_chk.MOSI;
112                 count_rx_ref--;
113             end
114         end
115         READ_DATA: begin
116             if (seq_item_chk.SS_n) begin
117                 cs_ref = IDLE;
118                 tx_valid_ref = 0;
119                 if (count_tx_ref == 0)
120                     rd_flag_ref = 0;
121             end
122             if (count_rx_ref != 0) begin
123                 rx_data_ref [count_rx_ref - 1] = seq_item_chk.MOSI;
124                 count_rx_ref--;
125             end
126             else if ((count_tx_ref != 0) && (tx_valid_ref)) begin
127                 MISO_ref = tx_data_ref [count_tx_ref - 1];
128                 count_tx_ref--;
129             end
130         end
131     endcase
132 end
133
134 // RAM
135 if (!seq_item_chk.rst_n) begin
136     wa_ref = 0;
137     ra_ref = 0;
138     tx_data_ref = 0;
139     tx_valid_ref = 0;
140 end
141 else if (rx_valid_ref) begin
142     case (rx_data_ref [ADDR_SIZE + 1 : ADDR_SIZE])

```

```

124         count_rx_ref--;
125     end
126     else if ((count_tx_ref != 0) && (tx_valid_ref)) begin
127         MISO_ref = tx_data_ref [count_tx_ref - 1];
128         count_tx_ref--;
129     end
130 end
131 endcase
132 end
133
134 // RAM
135 if (!seq_item_chk.rst_n) begin
136     wa_ref = 0;
137     ra_ref = 0;
138     tx_data_ref = 0;
139     tx_valid_ref = 0;
140 end
141 else if (rx_valid_ref) begin
142     case (rx_data_ref [ADDR_SIZE + 1 : ADDR_SIZE])
143         2'b00: wa_ref = rx_data_ref [ADDR_SIZE - 1 : 0];
144         2'b01: mem_ref [wa_ref] = rx_data_ref [ADDR_SIZE - 1 : 0];
145         2'b10: ra_ref = rx_data_ref [ADDR_SIZE - 1 : 0];
146         2'b11: tx_data_ref = mem_ref [ra_ref];
147     endcase
148     tx_valid_ref = (&(rx_data_ref [ADDR_SIZE + 1 : ADDR_SIZE]));
149 end
150 else begin
151     tx_valid_ref = 0;
152 end
153
154 // rx_valid
155 rx_valid_ref = (((cs_ref == WRITE) || (cs_ref == READ_ADD) || ((cs_ref == READ_DATA) && (!rx_valid_in_RD))) && (count_rx_ref == 0));
156 if (count_rx_ref == (ADDR_SIZE + 2))
157     rx_valid_in_RD = 0;
158 else if (count_rx_ref == 0)
159     rx_valid_in_RD = 1;
160 endtask
161
162 function void report_phase (uvm_phase phase);
163     super.report_phase(phase);
164     `uvm_info("report_phase", $sformatf("Total successful transactions: %0d", correct_count), UVM_MEDIUM)
165     `uvm_info("report_phase", $sformatf("Total failed transactions: %0d", error_count), UVM_MEDIUM)
166 endfunction
167 endclass
168 endpackage
169

```

Activate Windows

Go to Settings to activate Windows.

3.12 Wrapper_Coverage

```
package wrapper_coverage_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"
import wrapper_seq_item_pkg::*;
import shared_pkg::*;

class wrapper_coverage extends uvm_component;
    `uvm_component_utils(wrapper_coverage)

    uvm_analysis_export #(wrapper_seq_item) cov_export;
    uvm_tlm_analysis_fifo #(wrapper_seq_item) cov_fifo;
    wrapper_seq_item seq_item_cov;

    covergroup cvgrp;
        W_cmd_cov_cp: coverpoint seq_item_cov.cmd iff (seq_item_cov.rst_n && ~seq_item_cov.SS_n) {
            bins wa_bin = {WA};
            bins wd_bin = {WD};
            bins ra_bin = {RA};
            bins rd_bin = {RD};
        }
        W_rst_cov_cp: coverpoint seq_item_cov.rst_n {
            bins reset_active = {0};
            bins reset_inactive = {1};
        }
        ssn_cov_cp: coverpoint seq_item_cov.SS_n {
            bins active = {0};
            bins inactive = {1};
        }
        MISO_cov_cp: coverpoint seq_item_cov.MISO {
            bins miso_0 = {0};
            bins miso_1 = {1};
        }

        cmd_miso_cross : cross W_cmd_cov_cp, MISO_cov_cp {
            // Legal combinations based on your FSM
            bins write_phase = binsof(W_cmd_cov_cp) intersect {WA, WD} &&
                binsof(MISO_cov_cp.miso_0);

            bins read_phase = binsof(W_cmd_cov_cp) intersect {RA, RD} &&
                binsof(MISO_cov_cp.miso_1);

            // All other combinations are illegal
            illegal_bins illegal_operations =
                (binsof(W_cmd_cov_cp) intersect {WA, WD} && binsof(MISO_cov_cp.miso_1)) ||
                (binsof(W_cmd_cov_cp) intersect {RA, RD} && binsof(MISO_cov_cp.miso_0));
        }
    endgroup
endclass
```

```
49
50
51     function new(string name = "wrapper_coverage", uvm_component parent = null);
52         super.new(name, parent);
53         cvgrp = new;
54     endfunction
55
56     function void build_phase(uvm_phase phase);
57         super.build_phase(phase);
58         cov_export = new("cov_export", this);
59         cov_fifo = new("cov_fifo", this);
60     endfunction
61
62     function void connect_phase(uvm_phase phase);
63         super.connect_phase(phase);
64         cov_export.connect(cov_fifo.analysis_export);
65     endfunction
66
67     task run_phase(uvm_phase phase);
68         super.run_phase(phase);
69         forever begin
70             cov_fifo.get(seq_item_cov);
71             cvgrp.sample();
72         end
73     endtask
74 endclass
75 endpackage
```

3.13 Wrapper_Agent

```
1 package wrapper_agent_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import wrapper_driver_pkg::*;
5 import wrapper_sequencer_pkg::*;
6 import wrapper_monitor_pkg::*;
7 import wrapper_config_obj_pkg::*;
8 import wrapper_seq_item_pkg::*;
9
10 class wrapper_agent extends uvm_agent;
11     `uvm_component_utils(wrapper_agent)
12
13     wrapper_sequencer sqr;
14     wrapper_driver drv;
15     wrapper_monitor mon;
16     wrapper_config_obj wrapper_cfg;
17     uvm_analysis_port #(wrapper_seq_item) agt_ap;
18
19     function new(string name = "wrapper_agent", uvm_component parent = null);
20         super.new(name, parent);
21     endfunction
22
23     function void build_phase(uvm_phase phase);
24         super.build_phase(phase);
25         if (!uvm_config_db #(wrapper_config_obj)::get(this, "", "W_CFG", wrapper_cfg))
26             uvm_fatal("build_phase", "Agent-unable to get the virtual interface of the wrapper from the uvm_config_db")
27
28         if (wrapper_cfg.active == UVM_ACTIVE) begin
29             sqr = wrapper_sequencer::type_id::create("sqr", this);
30             drv = wrapper_driver::type_id::create("drv", this);
31         end
32         mon = wrapper_monitor::type_id::create("mon", this);
33         agt_ap = new("agt_ap", this);
34     endfunction
35
36     function void connect_phase(uvm_phase phase);
37         mon.mon_ap.connect(agt_ap);
38         mon.wrapper_monitor_if = wrapper_cfg.wrapper_config_if;
39         if (wrapper_cfg.active == UVM_ACTIVE) begin
40             drv.wrapper_driver_if = wrapper_cfg.wrapper_config_if;
41             drv.seq_item_port.connect(sqr.seq_item_export);
42         end
43     endfunction
44 endclass
45 endpackage
```

3.14 Wrapper_Env

```
1 package wrapper_env_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import wrapper_agent_pkg::*;
5 import wrapper_coverage_pkg::*;
6 import wrapper_scoreboard_pkg::*;
7
8 class wrapper_env extends uvm_env;
9     `uvm_component_utils(wrapper_env)
10
11     wrapper_agent agt;
12     wrapper_scoreboard sb;
13     wrapper_coverage cov;
14
15     function new(string name = "wrapper_env", uvm_component parent = null);
16         super.new(name, parent);
17     endfunction
18
19     function void build_phase(uvm_phase phase);
20         super.build_phase(phase);
21         agt = wrapper_agent::type_id::create("agt", this);
22         sb = wrapper_scoreboard::type_id::create("sb", this);
23         cov = wrapper_coverage::type_id::create("cov", this);
24     endfunction
25
26     function void connect_phase(uvm_phase phase);
27         super.connect_phase(phase);
28         agt.agt_ap.connect(sb.sb_export);
29         agt.agt_ap.connect(cov.cov_export);
30     endfunction
31 endclass
32 endpackage
```

3.15 Wrapper_Test

```
1 package wrapper_test_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import ram_config_pkg::*;
5 import slave_config_pkg::*;
6 import wrapper_config_obj_pkg::*;
7 import ram_env_pkg::*;
8 import slave_env_pkg::*;
9 import wrapper_env_pkg::*;
10 import wrapper_sequence_pkg::*;
11
12 class wrapper_test extends uvm_test;
13   `uvm_component_utils(wrapper_test)
14
15   ram_config R_cfg;
16   slave_config S_cfg;
17   wrapper_config_obj W_cfg;
18
19   ram_env R_env;
20   slave_env S_env;
21   wrapper_env W_env;
22
23   wrapper_reset_sequence rst_seq;
24   wrapper_write_only_sequence write_seq;
25   wrapper_read_only_sequence read_seq;
26   wrapper_write_read_sequence write_read_seq;
27
28   function new(string name = "wrapper_test", uvm_component parent = null);
29     super.new(name, parent);
30   endfunction
31
32   function void build_phase(uvm_phase phase);
33     super.build_phase(phase);
34     R_cfg = ram_config::type_id::create("R_cfg");
35     S_cfg = slave_config::type_id::create("S_cfg");
36     W_cfg = wrapper_config_obj::type_id::create("W_cfg");
37
38     R_env = ram_env::type_id::create("R_env", this);
39     S_env = slave_env::type_id::create("S_env", this);
40     W_env = wrapper_env::type_id::create("W_env", this);
41
42     rst_seq = wrapper_reset_sequence::type_id::create("rst_seq");
43     write_seq = wrapper_write_only_sequence::type_id::create("write_seq");
44     read_seq = wrapper_read_only_sequence::type_id::create("read_seq");
45     write_read_seq = wrapper_write_read_sequence::type_id::create("write_read_seq");
46
```

Activate Window

```
47   if (!uvm_config_db #(virtual ram_if)::get(this, "", "ram_vif", R_cfg.ram_vif))
48     `uvm_fatal("build_phase", "Test-unable to get the virtual interface of RAM")
49   R_cfg.active = UVM_PASSIVE;
50   uvm_config_db #(ram_config)::set(this, "", "CFG", R_cfg);
51
52   if (!uvm_config_db #(virtual slave_if)::get(this, "", "slave_vif", S_cfg.slave_vif))
53     `uvm_fatal("build_phase", "Test-unable to get the virtual interface of slave")
54   S_cfg.active = UVM_PASSIVE;
55   uvm_config_db #(slave_config)::set(this, "", "CFG", S_cfg);
56
57   if (!uvm_config_db #(virtual wrapper_if)::get(this, "", "wrapper_if", W_cfg.wrapper_config_if))
58     `uvm_fatal("build_phase", "Test-unable to get the virtual interface of wrapper")
59   W_cfg.active = UVM_ACTIVE;
60   uvm_config_db #(wrapper_config_obj)::set(this, "", "W_CFG", W_cfg);
61 endfunction
62
63 task run_phase(uvm_phase phase);
64   super.run_phase(phase);
65   phase.raise_objection(this);
66   `uvm_info("run_phase", "Reset Asserted", UVM_LOW)
67   rst_seq.start(W_env.agt.sqr);
68   `uvm_info("run_phase", "Reset Deasserted", UVM_LOW)
69
70   `uvm_info("run_phase", "Write-Only Sequence Started", UVM_LOW)
71   write_seq.start(W_env.agt.sqr);
72   `uvm_info("run_phase", "Write-Only Sequence Ended", UVM_LOW)
73
74   `uvm_info("run_phase", "Read-Only Sequence Started", UVM_LOW)
75   read_seq.start(W_env.agt.sqr);
76   `uvm_info("run_phase", "Read-Only Sequence Ended", UVM_LOW)
77
78   `uvm_info("run_phase", "Write-Read Sequence Started", UVM_LOW)
79   write_read_seq.start(W_env.agt.sqr);
80   `uvm_info("run_phase", "Write-Read Sequence Ended", UVM_LOW)
81   phase.drop_objection(this);
82 endtask
83 endclass
84 endpackage
85
```


3.16 Verification_Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
WRAP_REQ_1	On reset: wrapper drives Slave to IDLE, deasserts RAM controls, clears address/data latches, rx_valid=0, MISO idle/tri-state per spec.	Directed: power-on reset + mid-simulation reset pulses while SS_n toggles.	cover property p_wrap_rst; coverpoint cp_idle_after_reset; cp_miso_idle.	Assertions: a_wrap_rst_state, a_ram_ctrl_low_on_reset, a_no_rxvalid_on_reset.
WRAP_REQ_2	Opcode decode from Slave (WA/WD/RA/RD) routes to RAM interface correctly.	Constrained-random opcodes with directed single-frame per opcode.	Covergroup cp_cmd on MOSI[10:8]; cross (cmd × ssn_active).	Assertions: a_decode_WA_to_addr_latch, a_decode_WD_to_write, a_decode_RA_to_addr_phase, a_decode_RD_to_read.
WRAP_REQ_3	WRITE ADDRESS: capture 8-bit address from Slave payload; latch to RAM addr bus when frame completes.	Directed WA frames with walking 1/0 address patterns.	Coverpoints cp_addr_vals (bins: walking1/0, 0x00, 0xFF).	Scoreboard: address_shadow == RAM addr; Assertion a_addr_latched_on_latchbit.
WRAP_REQ_4	WRITE DATA: on WD, assert RAM WE, present latched address and data; one write per completed WD frame.	Directed WD frames after WA; random data patterns.	Coverpoints cp_we_pulse, cp_data_vals; cross(addr × data bucket).	Assertions: a_we_1cycle_at_frame_done, a_data_to_ram_matches_mosi; Scoreboard compares RAM model.
WRAP_REQ_5	READ ADDRESS: capture address and set received_address flag; no RAM access yet.	Directed RA frames (13-bit) with random payload.	Coverpoint cp_received_address_set; cp_cmd==RA bins hit.	Assertions: a_ra_sets_flag_only, a_no_we_no_re_on_ra.
WRAP_REQ_6	READ DATA (RX subpath): if tx_valid=0, Wrapper accepts incoming 8-bit data on MOSI (e.g., register write-back path) and pulses rx_valid.	Directed RD with tx_valid=0; randomized payload.	Coverpoint cp_tx_valid_off; cp_rx_valid_pulse_on_last; data pattern bins.	Assertions: a_rd_rx_captures_data; Scoreboard stores data to RAM model if design specifies.
WRAP_REQ_7	READ DATA (TX subpath): if tx_valid=1, Wrapper sources RAM read data to Slave/MISO for 8 clocks; clear received_address at end.	Directed RD with tx_valid=1; vary RAM contents; back-to-back reads.	Coverpoint cp_tx_valid_on; cp_miso_toggles; cross(cmd==RD × tx_valid=1).	Assertions: a_ram_read_kicks_on_rd, a_miso_drives_from_ram, a_clear_received_address.
WRAP_REQ_8	SS_n timing: normal transaction 1→0[*13]→1; extended read 1→0[*23]→1; mid-frame abort returns to IDLE and deasserts RAM controls.	Directed timing-accurate frames; plus deassert SS_n mid-frame in WA/WD/RA/RD.	cover property s_normal_frame, s_readext_frame; cp_abort_midframe.	Assertions: a_abort_to_idle_1cyc, a_ram_ctrl_low_on_abort.
WRAP_REQ_9	Counter/index safety: never index tx_data/ram_data out-of-range; clamp RD counter sB; no negative indexing.	Stress: long frames, late drop of tx_valid, jittered clocks.	Cover properties on counter ranges; cp_counter_values.	Assertions: a_counter_bounds, a_safe_indexing_no_neg1.
WRAP_REQ_10	FSM coverage: reach IDLE, CHK_CMD, WRITE, READ_ADD, READ_DATA; cover all legal transitions; default unreachable.	Directed op mix + random; self-loops while SS_n=0; ends with SS_n=1.	Tool FSM states & transitions report.	Assertions: a_state_enum_valid; coverage_off for default if proven unreachable.
WRAP_REQ_11	RAM bus protocol: addr stable when WE asserted; no WE and RE simultaneously; no RAM access when SS_n=1.	Directed sequences toggling SS_n and back-to-back ops; random address/data.	Coverpoints cp_we_re_exclusion; cp_addr_stable_during_we.	Assertions: a_no_we_and_re_a_addr_stable_when_we, a_no_ram_access_in_idle.
WRAP_REQ_12	End-to-end data integrity: WA→WD writes RAM; RA→RD returns the same data on MISO.	Directed sequences for several addresses with varied data, including 0x00/0xFF and walking patterns.	Coverpoints cp_e2e_scenarios; cross(addr bucket × data bucket).	Scoreboard: RAM reference model vs observed MISO; Assertions on equality.
WRAP_REQ_13	Back-to-back operations without idle: consecutive WD, consecutive RD (tx_valid=1), and mixed sequences.	Random bursts with minimal or zero SS_n gaps; directed pairs.	Coverpoints cp_b2b_types; transition bins (WA→WD, RA→RD, RD→RD, WD→WD).	Assertions: a_no_state_loss_b2b; a_correct_counter_reload_between_frames.
WRAP_REQ_14	Illegal/ignored combinations are not counted: e.g., RD with tx_valid=1 but RAM empty (if spec-defined), or WD without prior WA if forbidden.	Negative tests; randomized with constraints + explicit illegal attempts.	ignore_bins in crosses; illegal_bins for disallowed combos.	Assertions: a_block_illegal, a_flag_error_if_defined; Coverage waivers documented.
WRAP_REQ_15	Idle behavior between frames: MISO tri-state/idle, rx_valid stays 0, RAM controls deasserted.	Insert 1–2 cycle gaps between frames of varying lengths.	Coverpoints cp_miso_idle, cp_rx_valid_idle, cp_ram_ctrl_idle.	Assertions: a_idle_lines, a_no_spurious_activity.
WRAP_REQ_16	Sequential Domain Coverage: Ensure temporal behaviors (handshakes, frame lengths, aborts, last-bit strobes) are observed (non-vacuous).	Mirror key SVAs with cover property; drive antecedents explicitly.	SDC report: cover property hit counts; vacuity low.	Assertions: same properties without 'cover' for checking; ensure no vacuous passes.

3.17 Assertion_Table

Assertion Name	Description
wrap_rst_tx_valid_a	Ensures tx_valid is 0 when reset (rst_n) is active.
wrap_rst_rx_valid_a	Ensures rx_valid is 0 when reset (rst_n) is active.
wrap_rst_tx_data_a	Ensures tx_data is 0 when reset (rst_n) is active.
wrap_rst_rx_data_a	Ensures rx_data is 0 when reset (rst_n) is active.
wrap_rst_miso_a	Ensures MISO is 0 when reset (rst_n) is active.
wrap_miso_stable_a	Ensures MISO remains stable during non-read operations.
wrap_rx_valid_a	Ensures rx_valid asserts after 12–22 clock cycles of SS_n active.
wrap_tx_valid_a	Ensures tx_valid asserts for read operation after 12–22 cycles.
wrap_rx_data_a	Ensures rx_data matches the past MOSI bits when rx_valid is active.

3.18 Coverage_Report

/wrapper_coverag...	100.00%					
TYPE cvgrp	100.00%	100	100.00%		✓	auto(1)
+ CVP cvgrp:...	100.00%	100	100.00%		✓	
+ CVP cvgrp:...	100.00%	100	100.00%		✓	
+ CVP cvgrp:...	100.00%	100	100.00%		✓	
+ CVP cvgrp:...	100.00%	100	100.00%		✓	
+ CROSS cv...	100.00%	100	100.00%		✓	

Covergroup Coverage:					
Covergroups	1	na	na	100.00%	
Coverpoints/Crosses	5	na	na	na	
Covergroup Bins	9	9	0	100.00%	
Covergroup	Metric	Goal	Bins	Status	
TYPE /wrapper_coverage_pkg/wrapper_coverage/cvgrp	100.00%	100	-	Covered	
covered/total bins:	9	9	-		
missing/total bins:	0	9	-		
% Hit:	100.00%	100	-		
Coverpoint W_cmd_cov_cp	100.00%	100	-	Covered	
covered/total bins:	2	2	-		
missing/total bins:	0	2	-		
% Hit:	100.00%	100	-		
bin wa_bin	880	1	-	Covered	
bin wd_bin	952	1	-	Covered	
Coverpoint W_rst_cov_cp	100.00%	100	-	Covered	
covered/total bins:	2	2	-		
missing/total bins:	0	2	-		
% Hit:	100.00%	100	-		
bin reset_active	19	1	-	Covered	
bin reset_inactive	1964	1	-	Covered	
Coverpoint ssn_cov_cp	100.00%	100	-	Covered	
covered/total bins:	2	2	-		
missing/total bins:	0	2	-		
% Hit:	100.00%	100	-		
bin active	1832	1	-	Covered	
bin inactive	151	1	-	Covered	
Coverpoint MISO_cov_cp	100.00%	100	-	Covered	
covered/total bins:	2	2	-		
missing/total bins:	0	2	-		
% Hit:	100.00%	100	-		
bin miso_0	1975	1	-	Covered	
bin miso_1	8	1	-	Covered	
Cross cmd_miso_cross	100.00%	100	-	Covered	
covered/total bins:	1	1	-		
missing/total bins:	0	1	-		
% Hit:	100.00%	100	-		

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File (Line)	Hits	Status
/wrapper_top/DUT/slave/cover_prop_readdata_to_idle	SLAVE	Verilog	SVA	SPI_slave.v(252)	12	Covered
/wrapper_top/DUT/slave/cover_prop_readdata_hold	SLAVE	Verilog	SVA	SPI_slave.v(243)	188	Covered
/wrapper_top/DUT/slave/cover_prop_readadd_to_idle	SLAVE	Verilog	SVA	SPI_slave.v(234)	45	Covered
/wrapper_top/DUT/slave/cover_prop_readadd_hold	SLAVE	Verilog	SVA	SPI_slave.v(225)	637	Covered
/wrapper_top/DUT/slave/cover_prop_write_to_idle	SLAVE	Verilog	SVA	SPI_slave.v(216)	57	Covered
/wrapper_top/DUT/slave/cover_prop_write_hold	SLAVE	Verilog	SVA	SPI_slave.v(207)	593	Covered
/wrapper_top/DUT/slave/cover_prop_chk_to_read_data	SLAVE	Verilog	SVA	SPI_slave.v(198)	13	Covered
/wrapper_top/DUT/slave/cover_prop_chk_to_read_add	SLAVE	Verilog	SVA	SPI_slave.v(188)	52	Covered
/wrapper_top/DUT/slave/cover_prop_idle_hold	SLAVE	Verilog	SVA	SPI_slave.v(172)	149	Covered
/wrapper_top/DUT/slave/cover_prop_idle_to_chk	SLAVE	Verilog	SVA	SPI_slave.v(163)	132	Covered
/wrapper_top/DUT/slave/c_reset_outputs_low	SLAVE	Verilog	SVA	SPI_slave.v(133)	19	Covered
/wrapper_top/DUT/slave/c_counter_load_in_chk	SLAVE	Verilog	SVA	SPI_slave.v(139)	132	Covered
/wrapper_top/DUT/slave/c_ssn_rxvalid_write	SLAVE	Verilog	SVA	SPI_slave.v(145)	17	Covered
/wrapper_top/DUT/slave/a_ssn_rxvalid_read	SLAVE	Verilog	SVA	SPI_slave.v(149)	6	Covered
/wrapper_top/DUT/slave/c_ssn_rxvalid_read	SLAVE	Verilog	SVA	SPI_slave.v(153)	6	Covered

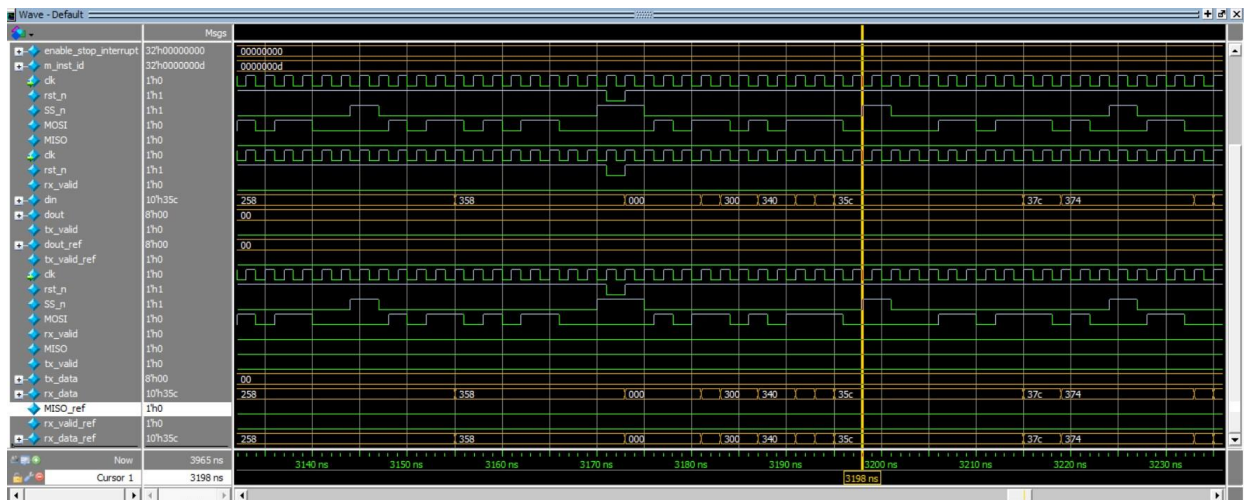
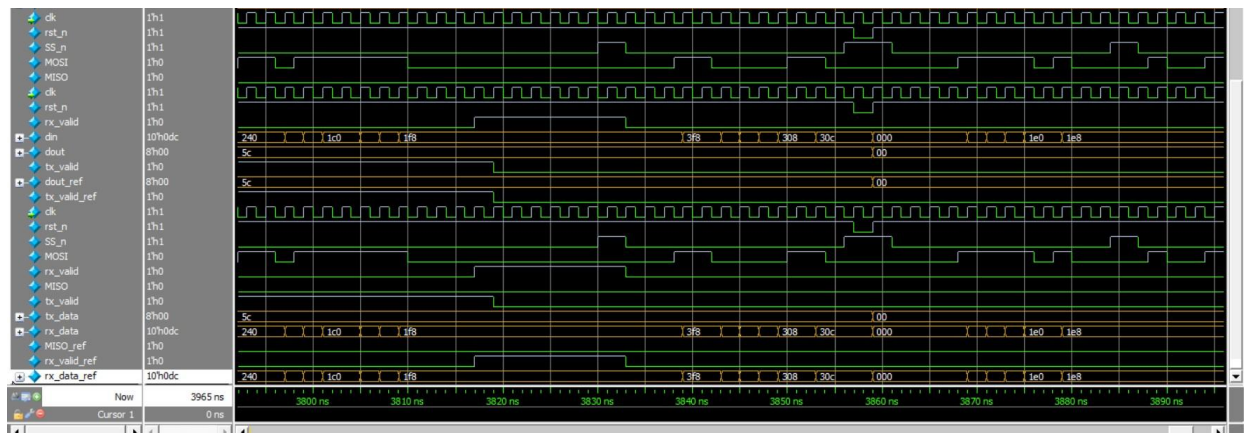
TOTAL DIRECTIVE COVERAGE: 100.00% COVERS: 15

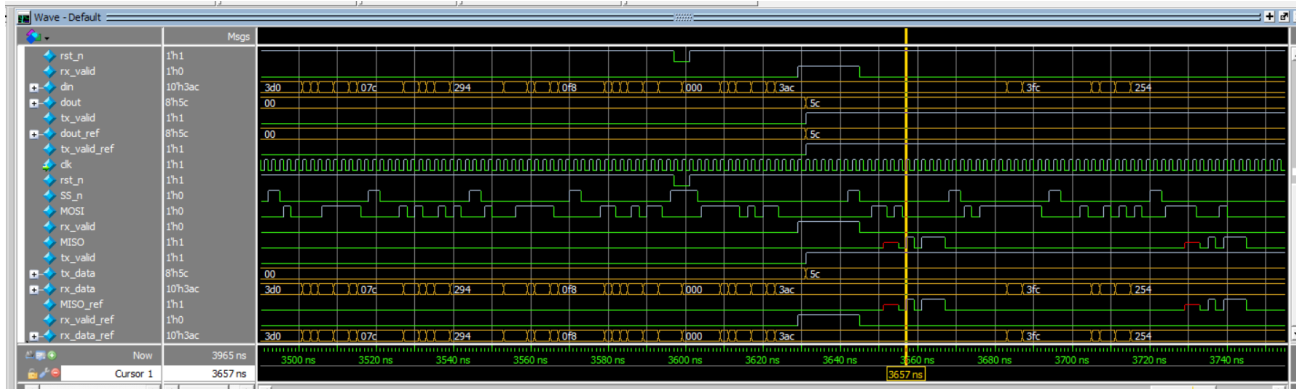
Total Coverage By Instance (filtered view): 89.23%

Cover Directives														
Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak Memory Time	Cumulative
✓ /wrapper_top/DUT... SVA	✓	Off	12	1	Unli...	1	100%		✓	0	0	0 ns		
✓ /wrapper_top/DUT... SVA	✓	Off	188	1	Unli...	1	100%		✓	0	0	0 ns		
✓ /wrapper_top/DUT... SVA	✓	Off	45	1	Unli...	1	100%		✓	0	0	0 ns		
✓ /wrapper_top/DUT... SVA	✓	Off	637	1	Unli...	1	100%		✓	0	0	0 ns		
✓ /wrapper_top/DUT... SVA	✓	Off	57	1	Unli...	1	100%		✓	0	0	0 ns		
✓ /wrapper_top/DUT... SVA	✓	Off	593	1	Unli...	1	100%		✓	0	0	0 ns		
✓ /wrapper_top/DUT... SVA	✓	Off	13	1	Unli...	1	100%		✓	0	0	0 ns		
✓ /wrapper_top/DUT... SVA	✓	Off	52	1	Unli...	1	100%		✓	0	0	0 ns		
✓ /wrapper_top/DUT... SVA	✓	Off	149	1	Unli...	1	100%		✓	0	0	0 ns		
✓ /wrapper_top/DUT... SVA	✓	Off	132	1	Unli...	1	100%		✓	0	0	0 ns		
✓ /wrapper_top/DUT... SVA	✓	Off	19	1	Unli...	1	100%		✓	0	0	0 ns		
✓ /wrapper_top/DUT... SVA	✓	Off	132	1	Unli...	1	100%		✓	0	0	0 ns		
✓ /wrapper_top/DUT... SVA	✓	Off	17	1	Unli...	1	100%		✓	0	0	0 ns		
✓ /wrapper_top/DUT... SVA	✓	Off	6	1	Unli...	1	100%		✓	0	0	0 ns		
✓ /wrapper_top/DUT... SVA	✓	Off	6	1	Unli...	1	100%		✓	0	0	0 ns		

Assertions												
Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threads	ATV	Assertion
▲ /uvm_pkg::uvm_re...	Immediate	SVA	on	0	0	-	-	-	-	-	off	assert (\$
▲ /uvm_pkg::uvm_re...	Immediate	SVA	on	0	0	-	-	-	-	-	off	assert (\$
▲ /wrapper_sequenc...	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert (r
▲ /wrapper_sequenc...	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert (r
▲ /wrapper_sequenc...	Immediate	SVA	on	0	1	-	-	-	-	-	off	assert (r
▲ /wrapper_top/DUT...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert((
▲ /wrapper_top/DUT...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert((
▲ /wrapper_top/DUT...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert((
▲ /wrapper_top/DUT...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert((
▲ /wrapper_top/DUT...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert((
▲ /wrapper_top/DUT...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert((
▲ /wrapper_top/DUT...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert((
▲ /wrapper_top/DUT...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert((
▲ /wrapper_top/DUT...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert((
▲ /wrapper_top/DUT...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert((
▲ /wrapper_top/DUT...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert((
▲ /wrapper_top/DUT...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert((
▲ /wrapper_top/DUT...	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert((

3.18 Wrapper_Questasim





```

Time: 3965 ns Iteration: 1 Region: /uvvm_pkg::uvvm_test_phase::execute
UVM_INFO wrapper_test.sv(80) @ 3965: uvvm_test_top [run_phase] Write-Read Sequence Ended
UVM_INFO verilog_src/uvvm-1.1d/src/base/uvvm_objection.svh(1267) @ 3965: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
UVM_INFO ram_scoreboard.sv(49) @ 3965: uvvm_test_top.R_env.sb [report_phase] total successful transactions:1982
UVM_INFO ram_scoreboard.sv(50) @ 3965: uvvm_test_top.R_env.sb [report_phase] total failed transactions:0
UVM_INFO slave_scoreboard.sv(49) @ 3965: uvvm_test_top.S_env.sb [report_phase] total successful transactions:1982
UVM_INFO slave_scoreboard.sv(50) @ 3965: uvvm_test_top.S_env.sb [report_phase] total failed transactions:0
UVM_INFO wrapper_scoreboard.sv(164) @ 3965: uvvm_test_top.W_env.sb [report_phase] Total successful transactions: 100
UVM_INFO wrapper_scoreboard.sv(165) @ 3965: uvvm_test_top.W_env.sb [report_phase] Total failed transactions: 0
--- UVM Report Summary ---
** Report counts by severity
UVM_INFO : 18
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0
** Report counts by id
[Questa UVM] 2
[RNIST] 1
[TEST_DONE] 1
[report_phase] 6
[run_phase] 8
** Note: $finish : C:/questasim64_2021.1/win64/verilog_src/uvvm-1.1d/src/base/uvvm_root.svh(430)
Time: 3965 ns Iteration: 61 Instance: /wrapper_top
1

```

3.19 Do_File

```

1
2 vlib work
3 vlog -sv -f src_list.list +define+SIM +cover -covercells
4 vsim -voptargs=+acc work.wrapper_top -cover
5 run 0
6 add wave *
7 add wave -position end sim:/wrapper_top/wrapperif/*
8 add wave -position end sim:/wrapper_top/ramif/*
9 add wave -position end sim:/wrapper_top/slaveif/*
10 coverage save wrapper_top.ucdb
11 run -all
12

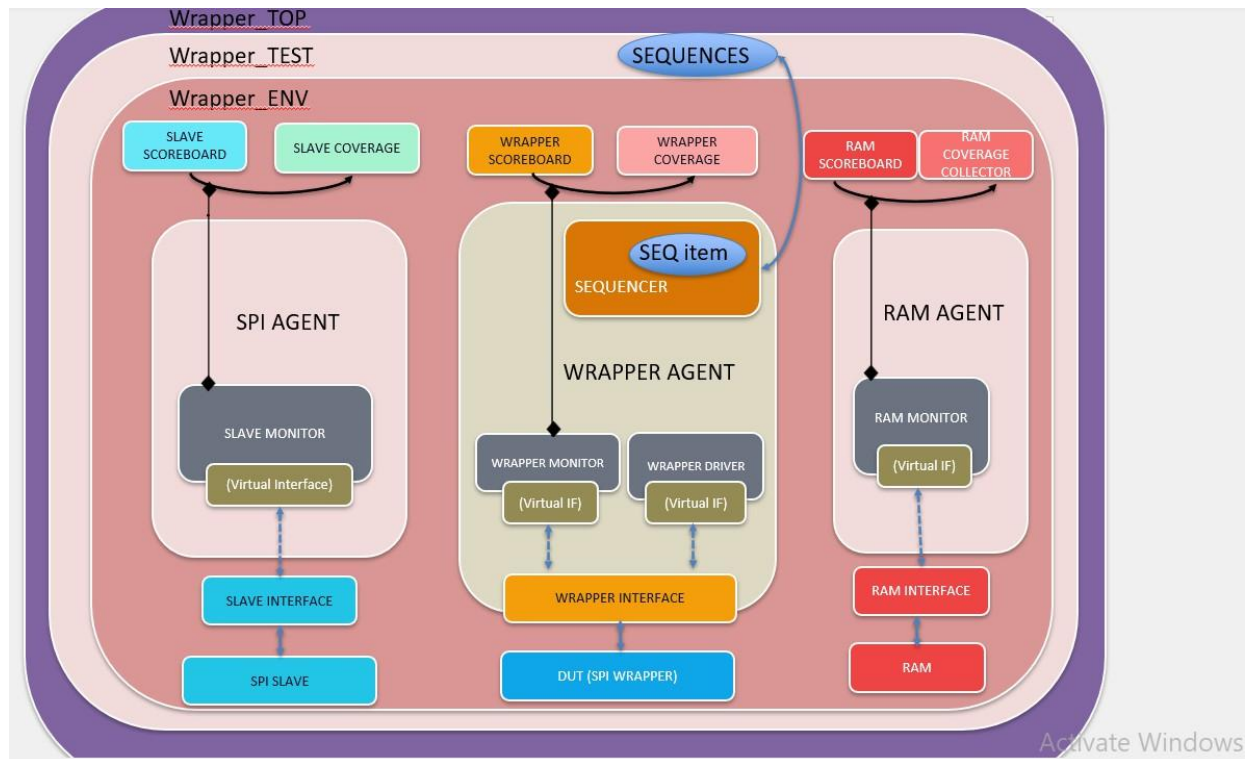
```

3.20 Scr_File

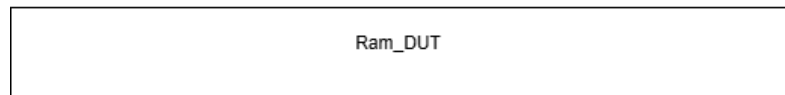
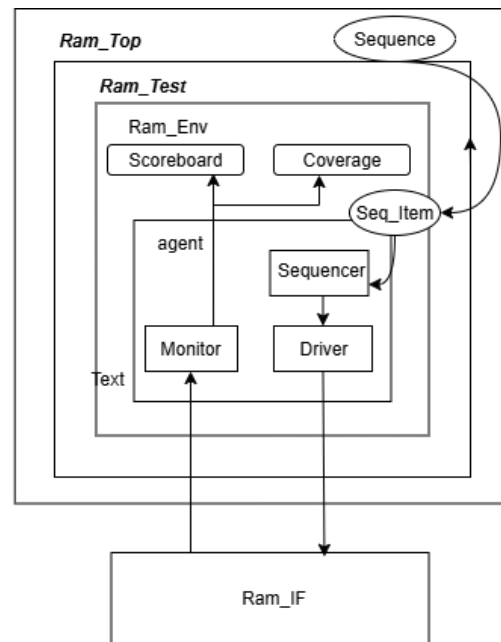
```
1  shared_pkg.sv
2  RAM.v
3  RAM_golden.v
4  ram_if.sv
5  ram_sva.sv
6  ram_sequence_item.sv
7  ram_sequence.sv
8  ram_sequencer.sv
9  ram_config.sv
10 ram_driver.sv
11 ram_monitor.sv
12 ram_scoreboard.sv
13 ram_coverage.sv
14 ram_agent.sv
15 ram_env.sv
16
17 SPI_slave.v
18 slave_golden.v
19 slave_if.sv
20 slave_seq_item.sv
21 slave_seq.sv
22 slave_sequencer.sv
23 slave_config_obj.sv
24 slave_driver.sv
```

```
23 slave_config_obj.sv
24 slave_driver.sv
25 slave_monitor.sv
26 slave_scoreboard.sv
27 slave_cov.sv
28 slave_agent.sv
29 slave_env.sv
30
31 wrapper.sv
32 wrapper_if.sv
33 wrapper_seq_item.sv
34 wrapper_sequence.sv
35 wrapper_sequencer.sv
36 wrapper_config_obj.sv
37 wrapper_driver.sv
38 wrapper_monitor.sv
39 wrapper_scoreboard.sv
40 wrapper_coverage.sv
41 wrapper_agent.sv
42 wrapper_env.sv
43 wrapper_test.sv
44 wrapper_top.sv
```

4.1 Wrapper



4.2 RAM



4.3 SLAVE

